

MLSYS · IM

First-Principles Infrastructure Modeling for Machine Learning Systems

Vijay Janapa Reddi

Harvard University

mlsysbook.ai/mlsysim

Abstract

As machine learning shifts from laboratory curiosity to critical infrastructure, the systems that sustain it span an extraordinary range, from sub-milliwatt microcontrollers to multi-gigawatt datacenter fleets. Reasoning across this range is hard: empirical profiling requires the target hardware in hand, while cycle-accurate simulation costs hours per configuration, leaving no tool for rapid, full-stack architectural reasoning. We present **MLSYS · IM** (Machine Learning Systems Infrastructure Modeling), a first-principles analytical framework that formalizes the “physics of systems” into a dimensionally-strict Python engine. MLSYS · IM is built on a *demand–supply* abstraction that decouples computational demand from silicon supply and environmental context, and it enforces unit integrity at runtime so the silent conversion errors that plague ad-hoc modeling cannot occur. Every input is drawn from a typed, provenance-tracked registry, so no number enters an analysis without a documented source. On this engine we codify a taxonomy of 22 “Systems Walls” resolved by 28 composable models and solvers, enabling sub-second design-space exploration that identifies binding constraints and synthesizes ideal hardware specifications across the entire ML systems lifecycle.

1 Introduction

Machine learning has become infrastructure (Sutton, 2019). Training a frontier model now requires orchestrating tens of thousands of accelerators across datacenter fabrics where memory ceilings, network bandwidth, power delivery, and regional carbon intensities interact in non-obvious ways (Dean et al., 2012; Shoeybi et al., 2019). The pace of this scaling is accelerating: frontier models have grown from billions to trillions of parameters in under five years, and the infrastructure cost of a single training run now rivals that of a small datacenter (DeepSeek-AI, 2025).

Yet the hardware required to develop intuition for these systems is prohibitively scarce: a student cannot requisition a 100,000-GPU cluster to explore how topology affects AllReduce latency, and a researcher cannot easily sweep parallelism strategies across hardware generations. This creates a growing *reasoning gap*. The systems are getting more complex, but the tools to think about them have not kept pace.

Consider a concrete example. A team deploying LLaMA-3 70B for interactive serving must answer: *How many H100 GPUs are needed to meet a 50 ms time-to-first-token SLA at 95th-percentile latency?* The answer depends on at least seven interacting constraints, each a “wall” (a hard bound imposed by physics, economics, or algorithmic scaling; see Table 1). We say a wall *binds* when it is the tightest constraint, the single bottleneck that limits end-to-end performance. The *binding constraint* is the wall whose relaxation would yield the largest throughput improvement; all other walls are slack.¹ For the LLaMA-3 serving example, the model’s 70 billion parameters require ~140 GB in FP16, exceeding a single GPU’s 80 GB HBM capacity (Wall 2: Memory). Tensor-parallel sharding across two GPUs introduces NVLink synchronization overhead (Wall 14: Communication). Continuous batching with PagedAttention determines KV-cache memory utilization (Wall 5: Batching). The decode phase is memory-bandwidth-bound at 3.35 TB/s per device (Wall 4: Serving). Tail latency under load follows Erlang-C queueing dynamics (Wall 7: Tail Latency). And the fleet’s total cost of ownership constrains what is economically viable (Wall 17: Capital). A similar multi-wall analysis applies to training: determining the optimal parallelism strategy for a 512-GPU run involves compute throughput, memory capacity, communication overhead, scaling laws, reliability, and economics simultaneously. No single equation suffices.

¹This usage mirrors linear programming: a constraint *binds* (is *active*) when the optimal point lies on it. Equivalently, in Roofline analysis, the binding ceiling is the one that determines achievable performance.

Existing tools are constrained along three axes (fidelity, speed, and scope), and no tool occupies the region where all three are adequate (Table 2). Profilers require physical silicon; cycle-accurate simulators like ASTRA-sim 2.0 (Won et al., 2023) require hours per configuration; analytical tools like Calculon (Isaev et al., 2023) achieve speed but focus narrowly on LLM training, ignoring data pipelines, reliability, sustainability, and inference. Patterson and Hennessy faced an analogous tradeoff in computer architecture education and chose taxonomic completeness over cycle accuracy: MIPS exposed every architectural concept through a model simple enough to reason about yet faithful enough to develop correct intuition (Hennessy et al., 2024). MLSYS·IM makes the same choice for ML systems. Beyond coverage gaps, none of these tools enforce dimensional correctness. In ML systems, confusing GB with GiB, FLOP/s with FLOPs, or bandwidth with throughput can silently invalidate capacity-planning spreadsheets, yet no existing tool catches these errors at runtime.

MLSYS·IM (Machine Learning Systems Infrastructure Modeling) is an open-source, pure-Python, first-principles analytical modeling framework for ML systems, designed for education and early design-space reasoning before empirical benchmarking. It formalizes back-of-the-envelope ML systems reasoning into a dimensionally strict, composable engine. The framework codifies 22 “Systems Walls” into 28 composable resolvers organized across six domains: Node, Data, Algorithm, Fleet, Operations, and Analysis. It separates computational *demand* from silicon *supply* and environmental *context* through a 5-layer demand–supply architecture, enforces SI unit correctness at runtime via the `pint` library, and produces full-stack analysis in milliseconds on any laptop.

Each of the resolvers computes structural physics from first-principles equations and hardware datasheet constants, then bridges the gap to measured performance through a single, explicit efficiency coefficient η that absorbs second-order effects (kernel launch overhead, straggler variance, framework dispatch costs). This is the same abstraction that makes the Roofline model useful: replacing peak bandwidth with *achievable* bandwidth sacrifices cycle-level precision for the ability to reason correctly about which constraint binds and why (Williams et al., 2009).

MLSYS·IM serves three communities. *Students* build quantitative intuition through hands-on tutorials where changing a single parameter reveals which wall binds and why. *Instructors* run live classroom demonstrations that produce concrete numbers in real time, replacing hand-waving with Roofline diagrams. *Researchers* perform rapid what-if analysis by comparing parallelism

strategies, evaluating procurement decisions, or projecting carbon footprints before committing resources. Designed as the analytical companion to the *Machine Learning Systems* textbook (Reddi et al., 2025a), MLSYS·IM is fully open-source (Apache-2.0 license) with deterministic, hardware-free execution, ensuring that every result in this paper is independently reproducible. This matters for educational equity. A student at a community college with a Chromebook can run the same quantitative exercises and verify the same binding constraints as one with access to a university GPU cluster. We validate against published empirical anchors spanning five taxonomy domains, matching values within 7% error while sweeping over 1,000 configurations in under one second.

This paper makes the following contributions:

- C1. A Taxonomy of 22 Systems Walls** (Table 1), each grounded in a published equation and resolved by a dedicated resolver (model or solver). The taxonomy provides a complete, structured vocabulary for reasoning about the constraints that bind ML system performance (Section 4).
- C2. Demand–Supply Separation with Dimensional Strictness.** A 5-layer architecture formally decouples computational demand from silicon supply through a single workload-lowering step. Every physical quantity carries SI units at runtime, transforming dimensional analysis from a manual discipline into a machine-checked invariant (Section 3).
- C3. Composable Resolver Algebra.** 28 resolvers (23 models, 2 solvers, and 3 optimizers) compose through chaining: each is a pure function $f(\text{config}) \rightarrow \text{metrics}$, producing a three-level evaluation (Feasibility, Performance, Macro) that identifies binding constraints. The algebra includes an inverse-Roofline *synthesis* solver that derives minimum hardware specifications from SLA requirements (Sections 5 and 7).
- C4. Accessible Full-Stack Reasoning without Hardware.** MLSYS·IM runs on any laptop without GPUs, clusters, or cloud credits. It powers deterministic labs and browser-friendly examples, providing an accessible systems engineering environment for students and instructors (Section 7).

The paper builds the framework in layers. We first survey the modeling landscape and identify the void (Section 2). Section 3 presents the architecture, including four design principles, a 5-layer input stack, the dimensionally strict type system, and the extensibility model. Section 4 formalizes the 22-wall taxonomy, and Section 5 defines the 3-tier resolver algebra (Models,

Table 1: **The 22 ML Systems Walls.** Each wall represents a physical or logical constraint resolved by a dedicated resolver (model or solver). Walls 1–2 (Compute and Memory) share the `SingleNodeModel`; four companion models deepen individual walls (`TrainingMemoryModel` for Wall 2, `ServingCapacityModel` for Wall 4, `MoERoutingModel` for Wall 14, and `NetworkRooflineModel`, which lifts the Roofline to fleet scale). Together with 3 optimizers (parallelism, batching, placement), the framework provides 28 resolvers across 22 walls. Domains progress from local node resources through data movement and algorithmic scaling to fleet coordination, operations, and cross-cutting analysis. Each wall is formalized in Section 4.

#	Wall	Resolver	Bounded	Core Equation	Ref.
<i>Node (Single-Accelerator Resources)</i>					
1	Compute	SingleNode	Peak FLOP/s	$T = \text{OPs} / (\text{Peak} \times \eta)$	Williams et al. 2009
2	Memory	SingleNode	HBM BW + cap.	$T = W / BW_{\text{HBM}}$	Williams et al. 2009
3	Software	Efficiency	Achieved MFU	$\eta = \text{FLOPS}_{\text{ach}} / \text{Peak}$	Chowdhery et al. 2023
4	Serving	Serving	Prefill vs. dec.	$T_{\text{pf}} = 2PS / (F\eta); T_{\text{dec}} = W / BW$	Pope et al. 2023
5	Batching	Cont. Batch	KV-cache frag.	$KV = 2LHD[S/p]pBb$	Kwon et al. 2023
6	Streaming	WeightStream	Injection BW	$T = \max(W_{\ell} / BW, 2P_{\ell}B / F\eta)$	Lie 2022
7	Tail Latency	TailLatency	P99 queueing	Erlang-C M/M/c	Dean and Barroso 2013
<i>Data (Movement & Pipelines)</i>					
8	Ingestion	Data	Storage I/O	$\rho = BW_{\text{demand}} / BW_{\text{supply}}$	Mohan et al. 2021
9	Transform.	Transform.	CPU preproc.	$T = B / R_{\text{cpu}}$	Murray et al. 2021
10	Locality	Topology	Bisection BW	$BW_{\text{eff}} = BW_{\text{link}} \cdot \beta / \text{osub}$	Leiserson 1985
<i>Algorithm (Scaling & Compression)</i>					
11	Complexity	Scaling	Scaling laws	$C = 6PD; P^* = \sqrt{C/120}$	Hoffmann et al. 2022
12	Reasoning	Inf. Scaling	Inf.-time comp.	$T = K \times T_{\text{step}}$	Snell et al. 2025
13	Fidelity	Compression	Acc.-efficiency	$r = b_{\text{base}} / b; r = 1 / (1-s)$	Han et al. 2016
<i>Fleet (Multi-Node Coordination)</i>					
14	Communic.	Distributed	AllReduce	$T = 2 \frac{N-1}{N} \frac{M}{B_{\text{link}}} + 2(N-1)\alpha$	Shoeybi et al. 2019
15	Fragility	Reliability	Cluster MTBF	$\text{MTBF}_{\text{cl}} = \text{MTBF}_{\text{node}} / N$	Daly 2006
16	Multi-tenant	Orchestration	Queue wait	$T_{\text{wait}} = \rho / [2\mu(1-\rho)]$	Little 1961
<i>Operations (Economics, Sustainability & Safety)</i>					
17	Capital	Economics	TCO	$\text{TCO} = \text{CapEx} + \text{OpEx}$	Barroso et al. 2018
18	Sustain.	Sustainability	Carbon + water	$\text{CO}_2 = E \times \text{PUE} \times \text{CI}$	Patterson et al. 2021
19	Checkpoint	Checkpoint	I/O burst penalty	$\text{penalty} = T_{\text{write}} / T_{\text{interval}}$	Eisenman et al. 2022
20	Safety	Resp. Eng.	DP-SGD overhead	$\sigma \propto 1/\epsilon$	Abadi et al. 2016
<i>Analysis (Cross-Cutting Diagnostics)</i>					
21	Sensitivity	Sensitivity	Binding constr.	$\partial T / \partial x_i$	Williams et al. 2009
22	Synthesis	Synthesis	Inverse spec	$BW_{\text{req}} = W / T_{\text{target}}$	Kwon et al. 2023
<i>Optimizers (Design-Space Search, no dedicated wall)</i>					
	Parallelism	Parallelism	Max MFU	$\max_c \eta(c)$ over $\text{TP} \times \text{PP} \times \text{DP}$	Shoeybi et al. 2019
	Batching	Batching	SLA-aware B^*	$\max B$ s.t. $P_{99} \leq T_{\text{SLA}}$	Kwon et al. 2023
	Placement	Placement	Cost-perf.	$\min \text{cost}$ s.t. $T \leq T_{\text{SLA}}$	—

Solvers, Optimizers) that composes over it. We validate against published benchmarks spanning five domains (Section 6), demonstrate the framework through student, instructor, and researcher use cases (Section 7), surface common misconceptions the framework is designed to expose (Section 8), discuss limitations and future work (Section 9), and conclude (Section 10).

2 Related Work

Tools for modeling and evaluating ML systems span a wide spectrum of fidelity, scope, and intended audience. We organize prior work into four categories and position `MLSYS·IM` relative to each: (1) cycle-level simulators that model hardware at the microarchitectural level, (2) accelerator design tools that evaluate individual chip

architectures, (3) analytical and co-design tools that trade fidelity for speed, and (4) pedagogical simulators that prioritize conceptual clarity. Table 2 provides a quantitative summary across these categories.

2.1 Cycle-Level Simulators

Cycle-accurate simulators provide the highest fidelity by modeling hardware behavior at the microarchitectural level. *gem5* (Binkert et al., 2011) is the canonical general-purpose architecture simulator, capable of modeling CPUs and GPUs down to individual pipeline stages. While invaluable for processor design, *gem5* lacks ML-specific abstractions (it has no notion of a transformer layer, a training step, or a parallelism strategy), and simulating even a single forward pass of a modern model can require hours of wall-clock time.

ASTRA-sim 2.0 (Won et al., 2023) addresses the ML gap by providing a hierarchical network simulator purpose-built for distributed training. It models collective communication patterns such as AllReduce across realistic network topologies, producing high-fidelity estimates of communication overhead. SimAI (Wang et al., 2025) extends this approach with a full-stack training simulator that integrates NS3-based network modeling with kernel computation traces, achieving 98% alignment with real-world results on 1024-node A100 clusters. Both inherit the fundamental cost of high fidelity. Simulating one training step of a large model at cluster scale can take minutes to hours, making iterative design-space exploration impractical. Their scope is also limited to communication and compute; neither models economics, sustainability, data pipelines, or reliability.

MLSYS·IM occupies a different point on the fidelity-speed spectrum. Where ASTRA-sim answers “how many microseconds does this AllReduce take on this exact topology,” MLSYS·IM answers “which of 22 possible bottlenecks binds this system, and how does changing hardware shift the binding constraint?” The two classes are complementary: MLSYS·IM narrows the design space, and high-fidelity simulators validate specific points within it.

2.2 Accelerator Design Tools

A second class of tools targets the design and evaluation of individual accelerator architectures. Timeloop (Parashar et al., 2019) provides a systematic methodology for evaluating DNN accelerator dataflows, modeling how data tiles map onto spatial architectures and estimating latency and energy for each mapping. Accelergy (Wu et al., 2019), its companion framework, supplies the energy estimation primitives that Timeloop consumes. Together, they form a powerful toolkit for accelerator architects exploring the design space

of novel silicon. LLMCompass (Zhang et al., 2024) brings this approach to LLM inference, combining an automated mapper with an area-based cost model to explore compute, memory bandwidth, and buffer configurations, achieving 4% error for end-to-end LLM inference on A100 nodes within minutes.

These tools operate at the operator and tile level, modeling how a single convolution or matrix multiplication executes on a specific microarchitecture. They do not reason about system-level concerns such as how multiple accelerators communicate across a network fabric, how the data pipeline feeds those accelerators, or what the total cost of ownership looks like at fleet scale. MLSYS·IM operates one abstraction level higher. It consumes the *outputs* of accelerator-level analysis (peak FLOP/s, memory bandwidth, TDP) as inputs to its hardware registry and reasons about how those specifications interact with workload demands, network topologies, and infrastructure constraints.

2.3 Analytical and Co-Design Tools

Closest in spirit to MLSYS·IM are analytical tools that sacrifice microarchitectural detail for speed. Calculon (Isaev et al., 2023) is an analytical co-design tool for large language model training. It models training time as a function of hardware specifications, parallelism strategies, and model architecture, achieving execution speeds comparable to MLSYS·IM. However, Calculon’s scope is narrow by design, targeting transformer-based LLM training exclusively, with no support for CNNs, mixture-of-experts architectures, or inference workloads. It does not model data pipelines, reliability, sustainability, economics, or safety considerations, and it lacks dimensional enforcement. Lumos (Liang et al., 2025) takes a trace-driven approach, using profiled kernel traces to predict LLM training performance at scale with 3.3% average error on up to 512 H100 GPUs. While Lumos achieves higher single-point accuracy than MLSYS·IM, it requires empirical traces from the target hardware, limiting its use in what-if exploration across hypothetical configurations. The DeepSeek-V3 systems paper (DeepSeek-AI, 2025) exemplifies the kind of hardware-aware co-design analysis that MLSYS·IM targets: it demonstrates how FP8 mixed-precision training, MoE sparsity, and multi-plane network topology interact to achieve frontier-model training at a fraction of conventional cost, a multi-wall optimization that spans Walls 1, 3, 13, 14, and 17 in our taxonomy.

Paleo (Qi et al., 2017) pioneered the analytical approach, decomposing DNN training time into computation and communication components across data- and model-parallel configurations. FlexFlow (Jia et al., 2019) optimizes parallelism strategies through simulation-

Table 2: **Comparison of ML systems modeling tools.** *Scope*: which system aspects a tool models (e.g., “compute & comm.” means compute throughput and collective communication only, while “full-stack” spans all six taxonomy domains). *Speed*: wall-clock time for a single evaluation. *Walls*: how many of the 22 Systems Walls (Table 1) each tool addresses. *Phase*: whether training, inference, or both are modeled. *Dist.*: whether multi-node (multi-accelerator) distributed analysis is supported.

Tool	Approach	Scope	Speed	Walls	Phase	Dist.
<i>Cycle-level</i>						
gem5	Cycle-accurate	CPU/GPU microarchitecture	Hours	1–2	Both	–
ASTRA-sim 2.0	Cycle-accurate	Network collectives, topology	Hours	1–2	Train	✓
SimAI	Trace-driven	Full-stack distributed training	Minutes	2–3	Train	✓
<i>Accelerator design</i>						
Timeloop + Accelergy	Analytical	Accelerator dataflow & energy	Minutes	1–2	Infer	–
LLMCompass	Analytical	LLM inference HW design space	Minutes	2–3	Infer	–
<i>Analytical & co-design</i>						
Paleo	Analytical	DNN training compute & comm.	Seconds	2	Train	✓
Calculon	Analytical	LLM training performance	Seconds	2–3	Train	✓
Lumos	Trace-driven	LLM training perf. modeling	Seconds	2–3	Train	✓
Vidur	Empirical	LLM inference scheduling	Seconds	3–4	Infer	–
GenZ	Analytical	LLM inference platform design	Seconds	2–3	Infer	–
LLM-Viewer	Analytical	LLM inference memory/latency	Seconds	1–2	Infer	–
<i>Sustainability</i>						
LLMCarbon	Analytical	LLM carbon footprint (op. + embodied)	Seconds	1	Both	–
CodeCarbon	Empirical	Runtime energy & carbon tracking	Seconds	1	Both	–
MLSYS · IM	Analytical	Full-stack: compute, memory, network, data, scaling, reliability, econ., sustainability, safety	Sub-sec.	22	Both	✓

guided search, and Habitat (Yu et al., 2021) provides cross-hardware extrapolation of training performance using execution-time scaling curves. While these tools advance specific aspects of analytical modeling, they predate or do not address the full scope of modern concerns (inference serving, fleet economics, sustainability). Vidur (Agrawal et al., 2024b) extends analytical modeling to LLM *inference*, using operator-level profiling to build a fine-grained runtime estimator validated at less than 5% error across multiple LLMs and scheduling policies. GenZ (Bambhaniya et al., 2024) provides an analytical framework for LLM inference platform design that models multi-dimensional network topologies and serving optimizations. DistServe (Zhong et al., 2024) and Sarathi-Serve (Agrawal et al., 2024a) advance LLM serving through prefill-decode disaggregation and chunked-prefill scheduling respectively, demonstrating that the two-phase inference model (Equation (7)) requires increasingly sophisticated scheduling to achieve high goodput. All of these tools focus on inference performance in isolation; they do not model training, data pipelines, economics, or sustainability.

LLM-Viewer (Yuan et al., 2024) and llm-analysis (Li, 2023) provide lightweight memory and latency estima-

tion for transformer inference. These tools are useful for single-model profiling but do not extend to fleet-level reasoning, multi-tenant scheduling, or cross-domain constraint analysis.

A parallel line of work targets sustainability and fleet efficiency. LLMCarbon (Faiz et al., 2024) projects end-to-end carbon footprints (operational and embodied) for dense and MoE LLMs, validated within 8% of Google’s published figures. CodeCarbon (Lottick et al., 2019) provides empirical energy tracking at runtime via hardware power monitors. Wongpanich et al. (2025) introduce *ML Productivity Goodput* (MPG) as a fleet-level efficiency metric for warehouse-scale TPU clusters, demonstrating that traditional utilization metrics are insufficient for characterizing ML fleet performance across model, data, framework, compiler, and scheduling layers. These tools each address one or two domains and do not model the full cross-stack interactions that determine *why* a workload consumes so much energy.

MLSYS · IM generalizes the analytical approach to the full ML systems stack. Where each tool above models one or two domains, MLSYS · IM composes resolvers spanning all six (Table 1): compute, memory, serving, data pipelines, scaling laws, fleet coordination,

economics, sustainability, and responsible engineering. Runtime dimensional strictness (SI units, no bare physical floats) is the invariant of Section 3.1.2.

2.4 Pedagogical Precedents

MLSYS·IM draws direct inspiration from the tradition of pedagogical simulators in systems education. Patterson and Hennessy’s MIPS/SPIM simulator (Patterson and Hennessy, 2014) taught generations of students computer architecture not by replicating a production processor, but by providing a simplified model that made architectural concepts tangible through rapid experimentation. Similarly, xv6 (Cox et al., 2011) and MINIX (Tanenbaum and Woodhull, 2006) teach operating systems by stripping away production complexity to reveal core abstractions.

MLSYS·IM follows this pedagogical philosophy for the ML systems domain. Production ML infrastructure (spanning millions of lines of code across frameworks, compilers, schedulers, and orchestrators) is too complex for students to reason about directly. MLSYS·IM provides a controlled environment in which students can sweep hardware configurations, vary parallelism strategies, and observe how binding constraints shift, all in rapid iteration cycles. Its integration with the companion textbook (Reddi et al., 2025a) provides structured laboratory exercises with autogradable assessments, a capability absent from every research-oriented tool surveyed in Section 2.

3 Architecture

Figure 1 presents the end-to-end framework. This section unpacks the design in five parts. Section 3.1 introduces the four principles (analytical speed, dimensional strictness, taxonomic completeness, demand–supply separation) that constrain every subsequent decision. Section 3.2 translates those principles into a five-layer input stack of workloads, hardware, infrastructure, topology, and resolvers. Section 3.3 describes the dimensionally strict type system that enforces correctness at every API boundary, Section 3.4 explains how new workloads, hardware, and resolvers compose with the existing pipeline without modification, and Section 3.5 maps the public import surface and the presentation layer that sits outside the analytical engine.

3.1 Design Philosophy

The central question MLSYS·IM addresses is: *Where does the complexity of an ML system come from?* We argue that complexity arises from the non-linear interaction of constraints across six distinct domains (node resources, data movement, algorithms, fleet coordination, operations,

and cross-cutting analysis) and that an effective modeling tool must formalize *all six* simultaneously. Four design principles govern MLSYS·IM’s approach.

3.1.1 Analytical Speed over Cycle Accuracy

Analytical models that execute in sub-second time enable iterative design-space exploration that cycle-accurate simulators cannot support at comparable speed. By absorbing microarchitectural detail (cache hit rates, warp scheduling) into a single efficiency parameter η , MLSYS·IM achieves sub-second execution per solve. This enables sweeps over thousands of hardware–model–topology combinations in seconds, the kind of rapid “what-if” exploration that Hennessy et al. (2024) identify as essential for quantitative architectural reasoning.

To maintain rigor despite analytical simplification, MLSYS·IM enforces a “**No Magic Numbers**” invariant. Every registry value carries a typed `Provenance` record whose *kind* (datasheet, literature, industry report, convention, estimate, derived, illustrative, or heuristic) determines what evidence it must supply: evidence-backed kinds require a source URL and a verification date, while estimates and derived values require explanatory notes, and all records require a dated verification. The H100’s peak FP16 throughput is not a bare 989 floating-point literal; it is $989 * \text{TFLOPs} / \text{second}$, sourced from NVIDIA’s published datasheet (NVIDIA Corporation, 2023). Shared sources live once in a central catalog of 126 provenance records (`core.provenance_catalog`) that registry entries reference by key, and an audit tool (`python -m mlsysim.tools.audit_provenance`) walks every registry and fails on any value with missing or weak provenance. This discipline ensures that analytical speed does not come at the cost of reproducibility.

3.1.2 Dimensional Strictness as an Invariant

Dimensional consistency is a pervasive challenge in systems modeling. Mixing gigabits with gigabytes, or omitting the refractive index of fiber in latency calculations, are canonical failure modes that silently corrupt results. MLSYS·IM treats dimensional correctness not as a feature but as a **runtime invariant**, analogous to memory safety in Rust. The framework wraps every physical quantity using the `pint` unit library, and gives compute work its own base dimension: a FLOP is not a dimensionless count, so compute throughput can never silently add to, or convert into, a memory bandwidth — the category error at the heart of many back-of-the-envelope mistakes. Mixing any two of {compute rate, bandwidth, latency} raises a deterministic `DimensionalityError` before any computation proceeds, while the ratios that *should* compose still do: arithmetic intensity (FLOP/byte) di-

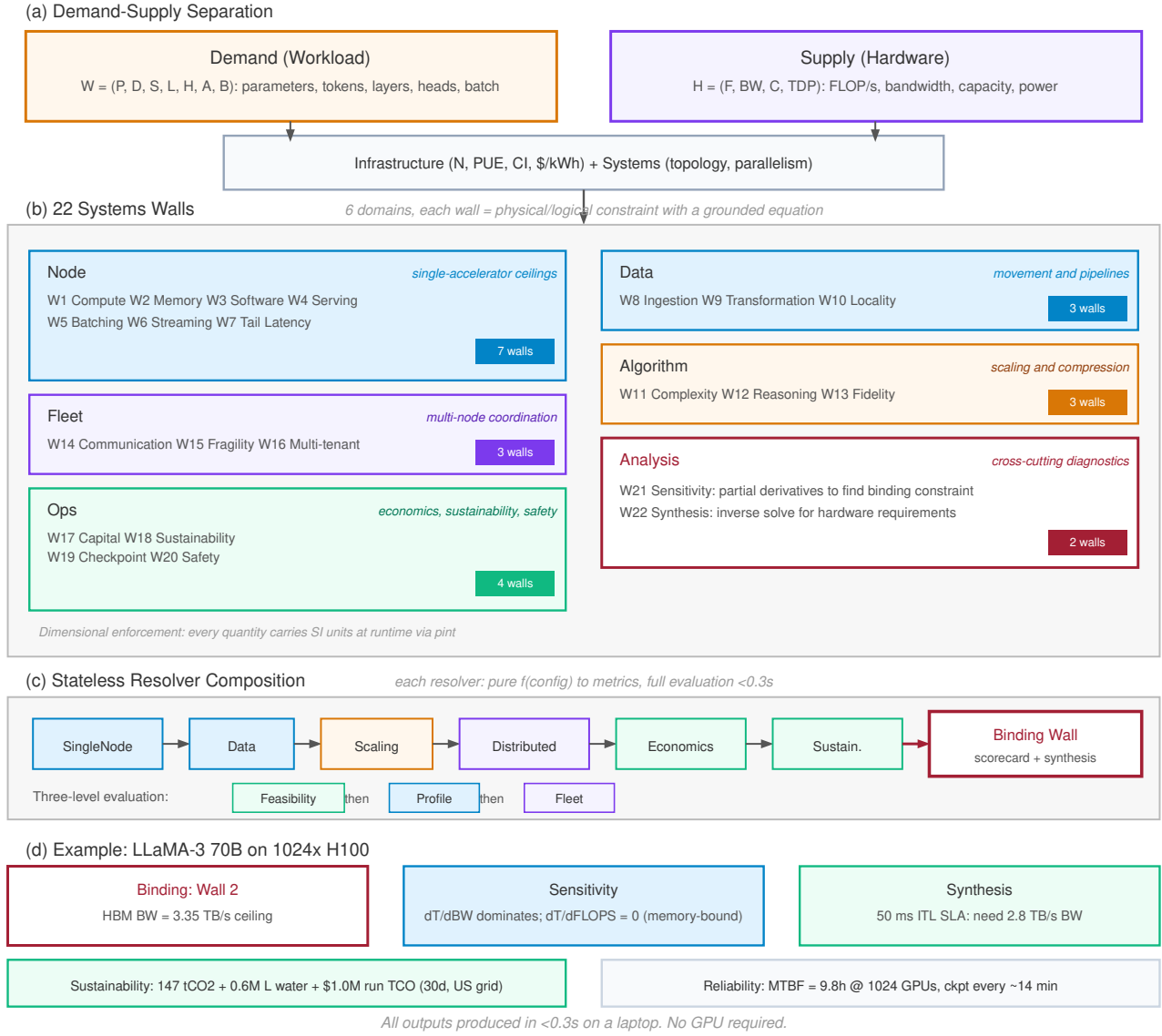


Figure 1: MLSys·IM Framework Overview. (a) Demand–supply separation decouples workload specifications from hardware capabilities and environmental context. (b) All 22 Systems Walls organized into six domains (Node, Data, Algorithm, Fleet, Operations, Analysis), each grounded in a published equation. (c) Stateless resolver composition chains 28 resolvers to identify binding constraints through a three-level evaluation. (d) Example outputs for LLaMA-3 70B on 1024× H100, produced in <0.3 s on a laptop.

vides cleanly, and utilization (achieved over peak FLOP rate) reduces to a dimensionless fraction:

```
1 from mlsysim.core.units import Q_
2 rate = Q_("989 TFLOPs/s") # Compute throughput
3 bw = Q_("3.35 TB/s") # Memory bandwidth
4 (rate / bw).to("flop/byte") # → 295 flop/byte (
5   ridge point)
6 rate + bw # → raises DimensionalityError
```

Listing 1: Dimensional Strictness. Prevents silent unit errors at the API level.

This design eliminates the single most common class of bugs in back-of-the-envelope systems analysis, namely silent unit conversion errors.² Hardware and model specifications live in typed registries (e.g., `Hardware.Cloud.H100.memory.bandwidth = 3.35 * TB/s`); the unit vocabulary lives in `core.units`, and physical constants and formulas live in `mlsysim.physics`, preventing unit mismatches

²The most infamous unit-conversion failure is the Mars Climate Orbiter, lost in 1999 because ground software produced thrust data in pound-force seconds while the spacecraft expected newton seconds (Stephenson et al., 1999).

at the API level. The physics formulas additionally validate the *dimensionality* of their arguments, raising `DimensionalityError` when, for example, a byte count is passed where a bandwidth is expected.

3.1.3 Taxonomic Completeness

We define a modeling framework as “complete” only when every fundamental bottleneck to scaling has a mathematical resolver. `MLSYS·IM` codifies 22 such bottlenecks, which we call *Systems Walls*, organized into six domains: Node (single-accelerator resources), Data (movement and pipelines), Algorithm (scaling and compression), Fleet (multi-node coordination), Operations (economics, sustainability, and safety), and Analysis (cross-cutting diagnostics). Each wall maps to a dedicated resolver with a formal equation grounded in the systems literature. Section 4 presents the full taxonomy with formal definitions, and Section 5 describes how solvers compose.

3.1.4 Demand–Supply Separation

`MLSYS·IM` enforces a strict separation between *what* a model computes and *where* it runs. A `TransformerWorkload` describes computational demand (parameters, layers, FLOPs, arithmetic intensity) without reference to any specific accelerator. A `HardwareNode` describes physical supply (peak throughput, memory bandwidth, TDP) without reference to any specific model. This decoupling, inspired by the compiler IR philosophy of separating representation levels (Hennessy et al., 2024), enables hardware–software co-design. The same GPT-3 workload can be evaluated against an H100, a TPU v5p, or a hypothetical future accelerator in a single parametric sweep, with all dimensional conversions handled automatically.

3.2 The Layered Input Stack

`MLSYS·IM` implements these design principles through a five-layer architecture (Figure 2). Layers A–D are independent *input layers* that describe demand, supply, context, and topology respectively; they do not depend on one another. The single lowering step occurs in Layer A, where a workload’s `lower()` method produces a hardware-agnostic `ComputationGraph` intermediate representation. Layer E (Resolvers) then consumes any combination of layers A–D as needed; a single-node analysis requires only A+B, while a fleet-wide carbon estimate draws on A+B+C+D.

Layer A: Workloads (Demand). A workload is a hardware-agnostic description of computational demand. `MLSYS·IM` provides five concrete workload types (Table 3), each exposing a `lower()` method that

Table 3: **Supported Workload Types.** Each workload lowers to a `ComputationGraph` with total FLOPs, weight bytes, and arithmetic intensity.

Workload	Key Parameters	Scaling
Transformer	P, L, H, D , seq. length	$2P$ FLOPs/token
CNN	P , inference FLOPs	Fixed per image
Sparse (MoE)	Total vs. active P , experts	Active P for FLOPs
SSM (Mamba)	P , state dim, D	$O(1)$ state cache
Diffusion	P , denoising steps T	$T \times$ FLOPs/step

produces a `ComputationGraph` (an intermediate representation containing total operations, weight bytes, and arithmetic intensity in `flop/byte`). This IR is the contract between demand and supply, capturing *what* must be computed without prescribing *how*.

Layer B: Hardware (Supply). A `HardwareNode` composes four subsystems: `ComputeCore` (peak FLOP/s with a precision-keyed dictionary for FP16, TF32, FP8, INT8), `MemoryHierarchy` (capacity and bandwidth), optional `StorageHierarchy`, and optional `IOInterconnect`. Each node also carries TDP, unit cost, and a kernel dispatch tax. The `ridge_point()` method computes the Roofline inflection $R = F_{\text{peak}} / BW_{\text{mem}}$ in `flop/byte` (Williams et al., 2009), enabling immediate classification of any lowered workload as compute-bound or memory-bound.

Layer C: Infrastructure (Context). `GridProfile` objects encode regional environmental parameters: carbon intensity (gCO_2/kWh), Power Usage Effectiveness (PUE), and Water Usage Effectiveness (WUE). A `Datacenter` composes a grid profile with rack-level power density constraints. This layer converts raw energy consumption into carbon footprint and water usage, following the methodology of Patterson et al. (2021).

Layer D: Systems (Topology). A `Fleet` composes `Nodes` (each a physical compute server containing one or more accelerators on PCIe or NVLink) with a `NetworkFabric` (topology, inter-node bandwidth, latency, oversubscription ratio). This layer enables distributed analysis. The `DistributedModel` decomposes workloads using 4D parallelism (data-parallel DP $\times$ tensor-parallel TP $\times$ pipeline-parallel PP $\times$ expert-parallel EP) and calculates hierarchical AllReduce costs, pipeline bubble fractions, and scaling efficiency.

Layer E: Resolvers (Analysis). Twenty-eight stateless resolvers (23 models, 2 solvers, and 3 optimizers) consume demand, supply, context, and topology to produce dimensioned performance metrics. The solver

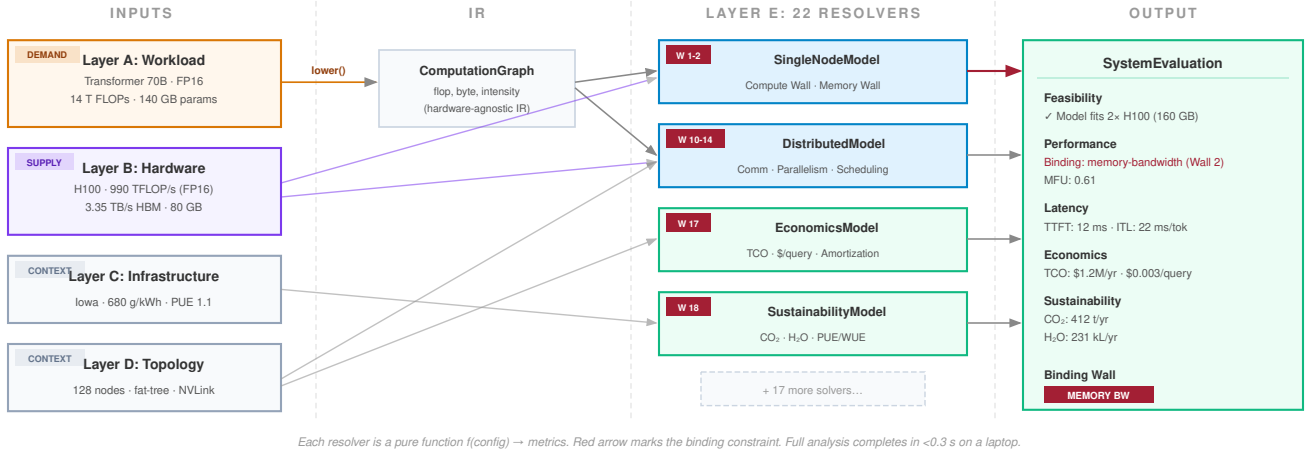


Figure 2: **The MLSys · IM 5-Layer Architecture.** Layers A–D provide typed inputs: workload demand, hardware supply, infrastructure context, and network topology. The single lowering step occurs in Layer A, where `lower()` produces a hardware-agnostic Computation Graph (an intermediate representation of total FLOPs, weight bytes, and arithmetic intensity). Layer E’s 28 stateless resolvers consume these inputs and produce a three-level SystemEvaluation scorecard. The red arrow marks the binding constraint identified by the solver chain.

formalism (stateless composition, chaining semantics, and three-level evaluation) is detailed in Section 5.

The five layers are useful only if their inputs are reproducible. MLSys · IM therefore organizes vetted specifications into eight curated registries collectively called the *MLSys Zoo*: **Hardware** (accelerators and boards, plus technology classes under `Hardware.Tech`), **Models** (workload architectures across six family files: language, vision, generative vision, recommendation, state-space, and TinyML), **Datasets** (benchmark catalog entries), **Platforms** (deployment envelopes), **Infrastructure** (grids, datacenters, racks, cooling, pricing, and capacity), **Systems** (nodes, racks, fabrics, clusters, pods, and storage paths), **Ops** (operational policies and overhead profiles such as training-run goodput budgets and monitoring thresholds), and executable **Scenarios** that compose existing model, hardware, infrastructure, and system entries with local constraints such as latency, power, run length, or region without redefining the underlying facts. Two auxiliary surfaces complete the picture: non-executable real-world anchors, such as Waymo data-rate ranges or mobile power envelopes, live in `ReferenceStats.*`, and cited field figures (MFU bands, scaling laws) live in `Literature.*`. Solver fallbacks live in `mlsysim.engine.calibration`; architecture formulas (Roofline, AllReduce, Young–Daly) live in `mlsysim.physics`; the machine-readable wall taxonomy of Table 1 ships as `mlsysim.engine.walls`; and domain-reviewed empirical anchors in `mlsysim.engine.empirical`

bind registry entries to published benchmark envelopes for regression testing.

Registry entries are authored as YAML data files and validated against `pydantic` schemas at import time, so stating a fact is a data edit while computing or composing remains Python. The loader rejects duplicate keys outright and resolves two reference forms: `@tech`: pointers that let an instance inherit a technology-class fact, and `@prov`: pointers into the shared provenance catalog (Section 3.1.1). The `Hardware.Tech` sub-registry encodes the load-bearing distinction between *instances* and *technology classes*: per-part facts that vary between products (capacity, bandwidth, TDP, price) live on instances such as `Hardware.Cloud.H100`, while facts that are properties of a technology generation (HBM access latency, energy per operation, interconnect latencies) live once on tech-class entries such as `Hardware.Tech.Memory.HBM3`, with instance lookups falling back to the tech class when unset. Each registry entry is a fully typed object; for example, `Hardware.Cloud.H100` returns a `HardwareNode` with all physical quantities dimensioned via `pint`. Figure 3 illustrates how these reusable facts become a concrete infrastructure model: a scenario binds a workload, accelerator fleet, regional grid, and operational policy to a local question, then passes the resulting typed objects to the resolver chain.

Once a scenario has assembled typed inputs, the `physics` package decomposes the framework’s analytical equations into nine domain-aligned modules (Table 4), each owning one conceptual domain of the systems stack, plus two supporting modules:

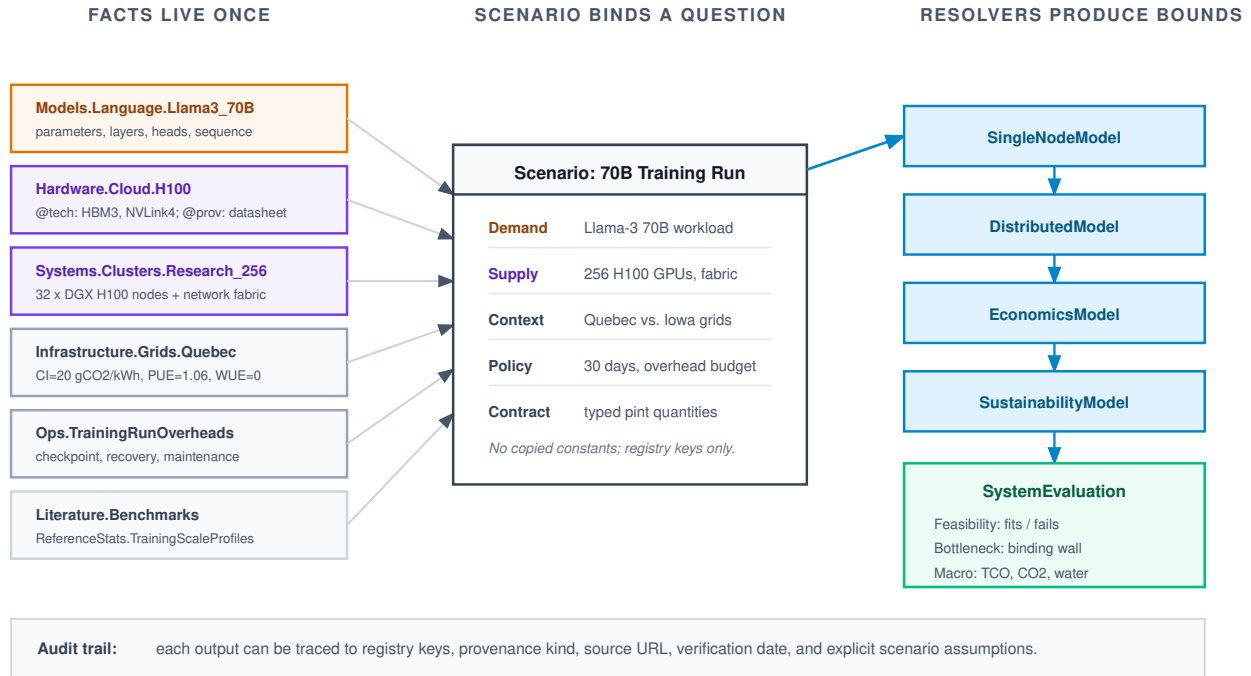


Figure 3: **Registry-Backed System Composition.** MLSYS · IM stores workload, hardware, systems, infrastructure, operational, and literature facts once, with typed schemas and provenance records. A scenario composes those reusable facts into a local infrastructure question—for example, training a 70B model on a 256-GPU H100 fleet in Québec or Iowa—and hands dimensioned objects to the resolver chain. The resulting `SystemEvaluation` can be traced back to the registry keys, source records, and scenario assumptions that produced it.

`physics.constants` holds universal physical constants (e.g., the speed of light in fiber), and `physics.quantities` provides generic dimensioned helpers (transfer time, compute time, energy from power, carbon from energy). A shared `_ensure_unit()` helper coerces raw floats into `pint` quantities at module entry points, guaranteeing dimensional correctness without burdening callers. Chapter and notebook code import from `mlsysim.physics` directly. The appendix (Table 9) lists the complete function signatures.

The **Hardware** registry holds 37 devices across five deployment tiers (Cloud, Workstation, Edge, Mobile, Tiny), spanning nine orders of magnitude in peak throughput from milliwatt-class microcontrollers (`Hardware.Tiny.ESP32_S3`, 520 KiB on-chip SRAM) to wafer-scale engines (`Hardware.Cloud.Cerebras_CS3`, 44 GB on-wafer SRAM, 125 PFLOP/s (Lie, 2022)). Beyond NVIDIA GPUs (T4 and V100 through B200 and the rack-scale GB200 NVL72), the registry includes AMD MI250X and MI300X, Intel Gaudi 2 and Gaudi 3, AWS Trainium 2, and the full Google TPU lineage (v1 through v6/Trillium), enabling cross-vendor design-space exploration, plus workstation (Apple

Table 4: **Physics Domain Modules.** Each module owns the analytical formulas for one taxonomy domain. All functions accept and return `pint` quantities.

Module	Domain	Key Formulas
performance	Node	Roofline bottleneck, Amdahl’s speedup, pipeline bubble
memory	Node	Model memory, activation memory, KV-cache, checkpoint size
serving	Node	Erlang-C M/M/c queue latency
communication	Fleet	Ring/tree/hierarchical AllReduce, All-to-All
reliability	Fleet	Young–Daly, cluster MTBF, failure probability
transformer	Algorithm	Training/decode FLOP counts
economics	Operations	Fleet TCO, egress cost
networking	Data	Speed-of-light latency
statistics	Analysis	PSI drift detection, sample sizing

M3 Max, DGX Spark), edge (Jetson, Coral), mobile (Apple A17 Pro, Snapdragon 8 Gen 3, Tensor G3), and TinyML (ESP32-S3, nRF52840, Himax WE-1) devices. The `list(sort_by=)` class method enables programmatic comparison, and users extend any zoo by instantiating new typed objects (Listing 2) or adding YAML data files, ensuring custom entries participate in the same dimensionally strict pipeline as vetted ones. Figure 4 maps the implementation structure behind this composition path: panel (a) shows package architecture (registries, physics modules, core utilities, and invariant checks), while panel (b) traces a concrete analysis from typed inputs through the resolver chain to the three-level `SystemEvaluation` scorecard. Appendix A gives a representative snapshot of the current registry surface.

3.3 The Type System

MLSYS-IM’s type system is built on Pydantic `BaseModel` classes with `pint` `Quantity` fields, providing both schema validation and dimensional enforcement at construction time. The same schemas validate the YAML registry data: every `*.yaml` entry is instantiated through its `pydantic` type at import, so a malformed unit string or a missing provenance record fails the import rather than surfacing later as a wrong number. The composition hierarchy is deliberately shallow: `HardwareNode` aggregates `ComputeCore` and `MemoryHierarchy` as direct fields, not through deep inheritance. This design makes the relationship between a hardware specification and its physical quantities immediately legible:

```
1 from mlsysim.hardware.types import *
2 from mlsysim.core.units import Q_
3 node = HardwareNode(
4     name="Custom Accelerator",
5     release_year=2025,
6     compute=ComputeCore(
7         peak_flops=Q_("500 TFLOPs/s"),
8         precision_flops={"fp8": Q_("1000 TFLOPs/s")
9     }
10 ),
11     memory=MemoryHierarchy(
12         capacity=Q_("96 GiB"),
13         bandwidth=Q_("4 TB/s")),
14     tdp=Q_("500 W"))
15 print(node.ridge_point()) # → 125 flop/byte
```

Listing 2: **Custom Hardware Node.** Composing a hardware specification with dimensional types.

The `ComputationGraph` IR bridges the demand-supply gap. When a solver calls `workload.lower()`, the workload computes its total operations, weight bytes, and arithmetic intensity, all in dimensioned quantities. For Mixture-of-Experts models, `SparseTransformerWorkload.lower()` uses

active parameters for FLOPs but *total* parameters for memory footprint, correctly modeling the fundamental decoupling between compute cost and capacity requirements in sparse architectures (Shazeer et al., 2017).

The complete evaluation produces a `SystemEvaluation` scorecard, a single object containing every metric from every resolver, cross-referenced by wall number. Students can inspect any individual wall or view the aggregate to understand how constraints interact across the full stack.

3.4 Extensibility

The layered architecture is designed for extension at every level. New workload types (e.g., a `RetrievalAugmentedWorkload` for RAG pipelines) require only implementing the `lower()` method to produce a `ComputationGraph`; all existing resolvers then apply without modification. New hardware entries are added to the **Hardware** registry as declarative `HardwareNode` specifications (Listing 2), with no resolver changes needed. New resolvers can be introduced for emerging constraints by implementing the appropriate tier interface: a Tier 1 Model (e.g., a `PrivacyModel` for federated learning overhead), a Tier 2 Solver, or a Tier 3 Optimizer.

By accepting typed inputs and returning dimensioned outputs, the type system enforces correctness at every boundary, ensuring that custom extensions compose safely with existing components. This design ensures that MLSYS-IM can track the rapidly evolving ML systems landscape without requiring architectural changes to the core framework.

3.5 Public API and Presentation Layer

The package separates analysis from presentation. `from mlsysim import *` exposes the registries, the physics formulas, the unit vocabulary, and the formatting helpers; solver classes are imported through the stable `mlsysim.solvers` path, which mechanically re-exports the canonical implementations in `mlsysim.engine.solvers` so the two surfaces cannot drift. The engine package is the infrastructure-modeling core: the Roofline core (`engine.engine`), the 28 resolvers, the three-level evaluation, pipeline composition, design-space exploration, executable scenarios, the machine-readable wall taxonomy, calibration defaults, and empirical anchors. Presentation lives outside the engine: `mlsysim.fmt` provides a typed `fmt_*` formatter family (50+ helpers for quantities, counts, currencies, percentages, and scientific notation) whose outputs are Markdown-safe strings and whose precision-safety guards refuse to render

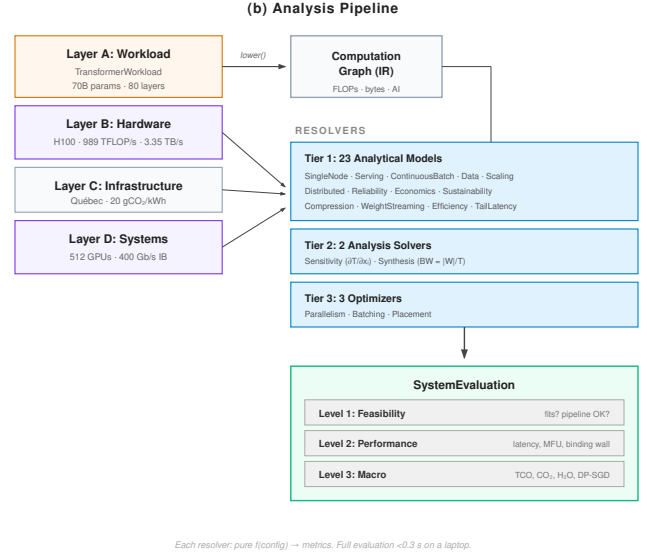
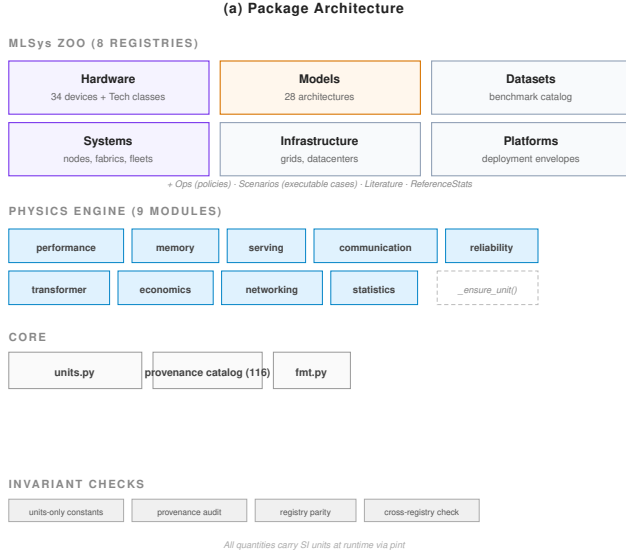


Figure 4: **System Anatomy.** (a) Package architecture: eight registries (six shown) provide typed, provenance-tracked specifications, nine physics modules supply domain-aligned formulas, and five invariant checks enforce design decisions as the codebase evolves. (b) Analysis pipeline: a workload lowers to a hardware-agnostic intermediate representation; four input layers feed 28 stateless resolvers organized in three tiers; the output is a three-level SystemEvaluation scorecard (Feasibility, Performance, Macro).

non-finite values or to silently round a non-zero value to the literal “0”; `mlsysim.viz` supplies matplotlib plotters with a consistent style; and a command-line interface (`python -m mlsysim`) exposes the zoo browser, scenario evaluation, serving analysis, design-space optimization, a local-hardware Iron Law audit, and schema export.

4 Taxonomy of ML Systems Walls

The ML systems literature is rich with specialized models for specific bottlenecks, from the original Roofline model (Williams et al., 2009) and Chinchilla scaling laws (Hoffmann et al., 2022) to PagedAttention batching limits (Kwon et al., 2023) and datacenter sustainability accounting (Patterson et al., 2021). However, these constraints are typically studied in isolation. We synthesize this disjointed literature into a unified taxonomy of 20 distinct “Walls” plus 2 cross-cutting diagnostic tools (Sensitivity and Synthesis). We borrow the term from the computer architecture tradition (the “memory wall” (Williams et al., 2009), the “power wall” (Hennessy et al., 2024)), where a *wall* denotes a hard physical or logical constraint that bounds system performance and cannot be circumvented by software optimization alone. Codifying these previously disparate equations into a single, composable framework, each wall is resolved by a dedicated resolver (model or solver) that accepts typed inputs and produces dimensionally correct bounds. This integration of theoretical and empirical constraints into

a single executable engine has not been done before. The complete taxonomy is summarized in Table 1.

The taxonomy proceeds outward from the silicon. Section 4.1 covers the seven Node walls that bound a single accelerator (compute, memory, software efficiency, serving, batching, streaming, tail latency). Section 4.2 adds the three Data walls that govern how samples reach the accelerator (ingestion, transformation, locality). Section 4.3 covers the three Algorithm walls that originate in the mathematics of learning (complexity, reasoning, fidelity). Section 4.4 adds the three Fleet walls that arise once a job spans multiple nodes (communication, fragility, multi-tenant scheduling). Section 4.5 covers the four Operations walls that determine whether a system should run at all (capital, sustainability, checkpoint, safety). Finally, Section 4.6 introduces the two cross-cutting diagnostic tools (Sensitivity and Synthesis) that operate across the entire taxonomy.

4.1 Node (Single-Accelerator Resources)

The Node walls define what a single accelerator (one physical processing unit such as a GPU, TPU, or wafer-scale engine, used interchangeably with “individual accelerator”) can achieve in isolation, before any distributed coordination is considered. Multi-node analysis, where a server hosts several such accelerators on intra-node links such as NVLink, falls under the Fleet domain (Section 4.4). Node walls are the innermost constraints and the first a practitioner should evaluate.

Table 5: **Notation.** Symbols used throughout; all quantities carry SI units at runtime via `pint`.

Symbol	Unit	Description
<i>Model & Workload</i>		
P, P_ℓ	params	Total / per-layer parameter count
$ W , W_\ell $	bytes	Bytes read per step / per layer
b_{prec}	B/param	Precision (e.g., 2 for FP16)
L, H, D	–	Layers, attention heads, head dim
S	tokens	Sequence length
B	samples	Batch size
K	–	Reasoning steps
C	FLOPs	Training compute ($6PD$)
I	FLOP/B	Arithmetic intensity
<i>Hardware & Infrastructure</i>		
$\text{Peak}_{\text{FLOPS}}$	FLOP/s	Peak accelerator throughput
BW_{HBM}	B/s	HBM bandwidth
BW_{inject}	B/s	Injection BW (wafer-scale)
BW_{link}	B/s	Per-link network bandwidth
N, G	–	Nodes in fleet, GPUs per node
α	s	Per-hop network latency
<i>Efficiency & Utilization</i>		
η	–	HW utilization ($\approx$ MFU)
η_{overlap}	–	Compute-comm overlap
ρ	–	Utilization ratio (queue/data)
$\beta, \beta_{\text{opt}}$	–	Bisection BW frac, optimizer multiplier
<i>Parallelism</i>		
TP, PP, DP, EP	–	Tensor, pipeline, data, expert parallel
V, M_{micro}	–	Virtual stages, microbatches
<i>Sustainability</i>		
PUE, WUE	–, L/kWh	Power / Water Usage Effectiveness
CI	gCO ₂ /kWh	Regional carbon intensity
<i>Key Derived Quantities</i>		
I^*	FLOP/B	Ridge point ($\text{Peak}/BW_{\text{HBM}}$)
B^*, P^*, D^*	varies	Optimal batch, model, dataset size
τ_{opt}	s	Optimal checkpoint interval

Wall 1: The Compute Wall. Every accelerator has a hard throughput ceiling determined by the number of arithmetic units and the clock frequency. An H100, for example, provides a peak of 989 TFLOP/s at FP16 with Tensor Cores, establishing an upper bound that no software optimization can exceed. The `SingleNodeModel` resolves this wall via Roofline analysis (Williams et al., 2009):

$$T_{\text{compute}} = \frac{\text{OPs}}{\text{Peak}_{\text{FLOPS}} \times \eta} \quad (1)$$

where $\eta \in (0, 1]$ is the hardware utilization efficiency and OPs is the total operation count. Throughout this paper, η denotes the ratio of sustained to peak throughput; the related metric *Model FLOPS Utilization* (MFU) measures only model-useful FLOPs and excludes overhead such as activation recomputation. For first-order analysis, we treat $\eta \approx$ MFU; the distinction matters only when recomputation or non-model compute is significant. When this wall binds, the only remedy is

faster silicon or fewer operations. **Assumptions:** Peak FLOPS is a hard ceiling; η is workload-dependent and must be specified or estimated from benchmark data.

Wall 2: The Memory Wall. High-bandwidth memory (HBM) imposes two ceilings: capacity (the model must fit) and bandwidth (weights must stream to compute units fast enough). An H100 reads HBM at 3.35 TB/s, yet its 989 TFLOP/s demand data at a rate that exceeds this bandwidth for any workload below ~ 295 flop/byte, making most LLM inference memory-bound, not compute-bound. During training, techniques like **Low-Rank Adaptation (LoRA)** and **Activation Recomputation** fundamentally alter the capacity constraint by trading compute or parameter trainability for drastically reduced memory footprints. The same `SingleNodeModel` computes (Williams et al., 2009):

$$T_{\text{memory}} = \frac{|W|}{BW_{\text{HBM}}} \quad (2)$$

where $|W|$ is the total bytes read per inference step. The realized execution time is the maximum of the two bounds:

$$T = \max(T_{\text{compute}}, T_{\text{memory}}) \quad (3)$$

The crossover between compute-bound and memory-bound regimes occurs at the *ridge point*, the arithmetic intensity at which the two ceilings intersect:

$$I^* = \frac{\text{Peak}_{\text{FLOPS}}}{BW_{\text{HBM}}} \quad (\text{flop/byte}) \quad (4)$$

Workloads with arithmetic intensity $I < I^*$ are memory-bound; those with $I > I^*$ are compute-bound. **Assumptions:** Peak FLOPS and HBM bandwidth are hard ceilings; MFU accounts for software inefficiency via η .

For training, capacity pressure is larger than inference weight residency alone. The `TrainingMemoryModel` exposes the first-order per-accelerator accounting directly:

$$M_{\text{train}} = M_{\text{weights}} + M_{\text{grad}} + M_{\text{opt}} + M_{\text{act}} + M_{\text{comm}}. \quad (5)$$

Mixed-precision Adam defaults to FP16/BF16 weights and gradients plus FP32 master weights and optimizer moments, while tensor, pipeline, expert, and ZeRO sharding reduce different terms of the sum (Shoeybi et al., 2019; Rajbhandari et al., 2020). Activation checkpointing reduces M_{act} by recomputing intermediate tensors during backward pass; the activation term implements the per-layer analytical bounds of Korthikanti et al. (2023) ($34sbh$ bytes per layer with selective recomputation; $34sbh + 5as^2b$ without, where s, b, h, a are sequence length, microbatch size, hidden dimension, and attention heads). The model intentionally reports

each term separately so students can see whether a configuration fails because of optimizer state, activations, or communication buffers rather than a generic “out of memory” label.

Wall 3: The Software Wall. The gap between peak and achieved FLOP/s is typically larger than the gap between hardware generations. Most naïve implementations achieve only 30% of peak throughput; the remaining 70% is lost to redundant memory traffic, low warp occupancy, and unfused operations. The `EfficiencyModel` models this as a multiplicative efficiency factor (Chowdhery et al., 2023):

$$\eta = \frac{\text{FLOPS}_{\text{achieved}}}{\text{Peak}_{\text{FLOPS}}} \quad (6)$$

where $\eta \in (0, 1]$ modulates the Roofline ceiling, reducing the effective peak from $\text{Peak}_{\text{FLOPS}}$ to $\eta \times \text{Peak}_{\text{FLOPS}}$. FlashAttention (Dao et al., 2022), for example, achieves a $2.5\times$ speedup over standard attention by fusing memory-bound operations into a single kernel pass, effectively raising η from ~ 0.3 to ~ 0.75 for attention layers. When this wall binds, better kernels, not bigger chips, are the remedy. **Assumption:** η is a single scalar that aggregates all software inefficiencies; in practice, different operations (GEMM vs. attention vs. normalization) achieve different utilization on the same silicon. To avoid circularity when η is unknown, the `EfficiencyModel` provides default ranges derived from published benchmarks: $\eta \approx 0.30\text{--}0.45$ for large-scale training (Chowdhery et al., 2023; Llama Team, AI Meta, 2024), $\eta \approx 0.50\text{--}0.60$ for highly optimized GEMM-heavy workloads, and $\eta < 0.10$ for memory-bound inference decode. Students can use these defaults as starting points and refine as they gather profiling data.

Wall 4: The Serving Wall. Autoregressive LLM inference exhibits two distinct phases with fundamentally different Roofline characteristics (Pope et al., 2023). For a 70B model on two tensor-parallel H100s, a 512-token prefill takes ~ 72 ms (compute-bound) while each decode token costs ~ 21 ms (memory-bound). Prefill processes all input tokens in parallel (~ 7066 tokens/s), while decode generates one token per memory-bandwidth pass (~ 48 tokens/s), a $\sim 148\times$ throughput gap. The `ServingModel` decomposes end-to-end inference latency as:

$$T_{\text{prefill}} = \frac{2P \cdot S_{\text{in}}}{\text{Peak}_{\text{FLOPS}} \times \eta} \quad (\text{compute-bound}) \quad (7)$$

$$T_{\text{decode}} = \frac{|W|}{BW_{\text{HBM}}} \quad (\text{memory-bound}) \quad (8)$$

where S_{in} is the input sequence length, P is the parameter count, and $|W|$ includes both model weights

and KV-cache reads (which grow with batch size and context length: $|W| = |W_{\text{model}}| + |W_{\text{KV}}|$). The $2PS$ term accounts for the linear projection FLOPs. For long contexts ($S > 4096$), the self-attention computation adds $\mathcal{O}(S^2)$ FLOPs per layer, which can constitute 20–40% of total prefill cost. The `ServingModel` includes this attention term explicitly. When serving multiple concurrent requests ($B > 1$), model weights are loaded from HBM once per step while KV-cache reads grow linearly with the batch: $T_{\text{step}} = (|W| + B \cdot |KV_1|) / BW$, where $|KV_1|$ is the per-request KV-cache size. Because each step produces B tokens simultaneously, the effective per-token cost is T_{step} / B , amortizing the weight read across requests. At large batch sizes the per-request KV cache dominates and throughput saturates.

The model also exposes an optional chunked-prefill estimate inspired by Sarathi-Serve (Agrawal et al., 2024a). When `prefill_chunk_tokens` is set, only the new, uncached prefill work is partitioned into chunks:

$$C_{\text{prefill}} = \max\left(1, \left\lceil \frac{S_{\text{in}} - S_{\text{cache}}}{S_{\text{chunk}}} \right\rceil\right), \quad (9)$$

$$T_{\text{stall}} = \max_i (T_{\text{chunk},i}) \quad (10)$$

where $T_{\text{chunk},i}$ is the compute time for chunk i plus one dispatch tax. The resulting `ServingResult` reports `prefill_chunks` (C_{prefill}), `prefill_chunk_time` (T_{stall}), and `decode_stall_bound`, which is set to T_{stall} as a first-order bound on how long the slowest prefill chunk can interfere with ongoing decode iterations. This is a coarse analytical bound, not a full scheduler: total TTFT still includes the same prefill compute work plus one dispatch tax per chunk, and the solver does not claim Sarathi-Serve’s stall-free packing behavior.

For deployment sizing, the serving-capacity resolver combines serving latency, continuous-batching capacity, and tail-queueing estimates. Given average generated length G and per-replica token throughput R_{tok} , it estimates request capacity as

$$Q_{\text{replica}} = \frac{R_{\text{tok}}}{G}, \quad (11)$$

$$P99_{\text{request}} \approx T_{\text{base}} + P99_{\text{queue}} \leq T_{\text{target}}. \quad (12)$$

This converts a common serving question—“how many replicas do I need for a QPS and P99 target?”—into a transparent composition of the latency, batching, and queueing walls. The compute efficiency η remains an exposed parameter with the same interpretation as in the Roofline model; the default is a starting point, not a claim of universal serving efficiency.

The prefill phase processes all input tokens in parallel and is compute-bound; the decode phase generates

one token at a time and is memory-bandwidth-bound. The solver incorporates modern paradigms including **Prompt Caching** (prefix caching (Zheng et al., 2024), which reduces TTFT by skipping prefill for previously computed KV-cache entries), **Speculative Decoding** (probability-weighted verification using a smaller draft model (Leviathan et al., 2023)), **Disaggregated Serving** (phase splitting onto different hardware with KV-cache network transfer (Patel et al., 2024)), and **Chunked Prefill** (optional chunk-level stall bounds for interleaving prefill work with decode (Agrawal et al., 2024a)). This duality explains why batching strategies that improve prefill throughput may have no effect on decode latency, because the two phases are bound by different resources. **Assumptions:** Prefill is compute-bound for sequence lengths $S_{\text{in}} \gg 1$; decode is memory-bandwidth-bound at batch size 1. Chunked prefill partitions the same upper-bound prefill work by token count with a fixed dispatch tax per chunk, so `decode_stall_bound` is a scheduling proxy rather than a measured tail-latency guarantee. At large batch sizes, decode transitions toward compute-bound; the solver models this crossover via the Roofline.

Wall 5: The Batching Wall. Serving throughput depends on how many requests share the accelerator simultaneously, but memory-bound decode means that each additional request in the batch adds KV-cache pressure without reducing per-token latency. Static batching wastes memory through external fragmentation. Each request reserves a contiguous KV-cache block sized for maximum sequence length, even if most requests finish early. The `ContinuousBatchingModel` models iteration-level scheduling with non-contiguous allocation via `PagedAttention` (Kwon et al., 2023):

$$KV_{\text{paged}} = 2 \times L \times H \times D \times \lceil S/p \rceil \times p \times B \times b \quad (13)$$

where L is layers, H is KV heads, D is head dimension, S is sequence length, p is page size in tokens, B is batch size, and b is bytes per element. Internal fragmentation is bounded by the last page, eliminating the 40–50% external fragmentation of contiguous allocation. **Assumptions:** Decode is memory-bandwidth-bound for batch ≥ 1 ; static batching baseline assumes 50% fragmentation waste.

Wall 6: The Streaming Wall. Wafer-scale architectures (Lie, 2022) (e.g., Cerebras CS-3) invert the conventional memory hierarchy. Activations reside on-wafer in SRAM while model weights stream from external MemoryX nodes, shifting the bottleneck from HBM bandwidth to injection interconnect bandwidth. The `WeightStreamingModel` models this as:

$$T_{\text{layer}} = \max \left(\frac{|W_\ell|}{BW_{\text{inject}}}, \frac{2P_\ell \times B}{\text{Peak} \times \eta} \right) \quad (14)$$

where $|W_\ell|$ is the layer weight size in bytes, $P_\ell = |W_\ell|/b_{\text{prec}}$ is the parameter count (with b_{prec} bytes per element), and the factor of 2 accounts for the multiply-accumulate FLOPs per parameter. The two terms inside the max represent the injection time (weight delivery) and the compute time (matrix arithmetic) for one layer. When B is small, weight injection dominates and the compute engine sits idle; when B is large, compute dominates and the injection link sits idle. Setting the two terms equal and substituting $|W_\ell| = P_\ell \times b_{\text{prec}}$ yields the optimal batch size $B^* = (b_{\text{prec}} \times \text{Peak} \times \eta) / (2 \times BW_{\text{inject}})$. This result depends only on numerical precision, peak compute, and injection bandwidth, independent of layer size. This is the unique operating point where injection and compute perfectly overlap, maximizing utilization of both resources. **Assumptions:** Layer weights dominate the injection payload; 10% overhead is reserved for working memory; perfect within-layer pipelining is assumed.

Wall 7: The Tail Latency Wall. At scale, P99 latency governs user experience, not the median. A single slow replica in a fan-out of 100 services dominates end-to-end response time. The `TailLatencyModel` models inference replicas as an M/M/c queue using the Erlang-C formula (Dean and Barroso, 2013):

$$\mathbb{P}[\text{wait}] = \frac{(c\rho)^c / c! \cdot (1 - \rho)^{-1}}{\sum_{k=0}^{c-1} (c\rho)^k / k! + (c\rho)^c / c! \cdot (1 - \rho)^{-1}} \quad (15)$$

where c is the number of replicas, $\rho = \lambda / (c\mu)$ is per-server utilization, and λ, μ are arrival and service rates. The implementation evaluates the Erlang-C sum in log space (a log-sum-exp over log-gamma terms), so deep replica pools do not overflow the factorials in Equation (15). P99 latency grows non-linearly as $\rho \rightarrow 1$: for a 10-replica deployment, P99 at 80% utilization is $\sim 3 \times$ the service time; at 95%, it rises to $\sim 10 \times$, making the distinction between the two the difference between meeting and violating a latency SLA. **Assumption:** The M/M/c model is intentionally optimistic; real traffic is bursty and service times are heavy-tailed, so M/M/c provides a lower bound on tail latency. If the system is unstable under M/M/c, it will certainly be unstable under real traffic. To bridge toward more realistic service-time distributions, the solver accepts a `service_time_cv` parameter (coefficient of variation); when $cv \neq 1$, a Kingman correction factor $(c_v^2 + 1)/2$ scales the wait time, capturing the heavier tails of non-exponential service times.

4.2 Data (Movement & Pipelines)

The Data walls govern how data moves to the accelerator. A node that is locally unconstrained can still starve if the data pipeline cannot keep pace.

Wall 8: The Ingestion Wall. Storage I/O must supply training samples at the rate the accelerator consumes them. This wall binds most often in vision tasks, where raw images are large. For example, an 8-GPU DGX A100 training ResNet-50 at batch 2,048 demands ~ 2 GB/s of compressed image reads per step, easily saturating a single NVMe SSD and requiring striped or cached storage. The `DataModel` computes the demand-supply ratio (Mohan et al., 2021):

$$\rho_{\text{data}} = \frac{BW_{\text{demand}}}{BW_{\text{supply}}} \quad (16)$$

When $\rho_{\text{data}} > 1$, the accelerator stalls waiting for data. BW_{demand} is the product of batch size, sample size, and step rate; BW_{supply} is the effective throughput of the storage subsystem after accounting for read amplification and caching. For LLM training on tokenized text, BW_{demand} is typically negligible (< 1 MB/s); this wall is primarily relevant for vision, audio, and video workloads. **Assumption:** Storage bandwidth is the bottleneck, not network I/O (single-node training).

Wall 9: The Transformation Wall. JPEG decoding, tokenization, and augmentation execute on CPU cores, not on the accelerator. When the CPU pre-processing pipeline cannot keep pace, the accelerator stalls even if storage bandwidth is abundant. On ImageNet, a single CPU worker typically decodes and augments ~ 850 images/s; 64 workers therefore cap the pipeline at $\sim 54,400$ images/s, below what an 8-GPU DGX A100’s compute can consume (see Case S2, Section 7). The `TransformationModel` quantifies this bottleneck (Murray et al., 2021):

$$T_{\text{transform}} = \frac{B}{R_{\text{cpu}}} \quad (17)$$

where B is the batch size (in samples) and R_{cpu} is the aggregate CPU preprocessing rate (in samples/s), computed as $R_{\text{cpu}} = N_{\text{workers}} \times r_{\text{worker}}$ where r_{worker} is the per-worker throughput after all transformations (decode, augment, normalize). **Assumption:** Preprocessing is CPU-bound and scales linearly with worker count up to core saturation.

Wall 10: The Locality Wall. Network topology determines the effective bandwidth available between any two nodes in the cluster. The `TopologyModel` models this through the *bisection bandwidth fraction* β (Leiserson, 1985), which varies by topology: Fat-Tree provides full bisection ($\beta = 1.0$), Dragonfly achieves $\beta \approx 0.85$, and 3D Torus yields $\beta \approx 0.67$. The effective inter-node bandwidth is:

$$BW_{\text{eff}} = \frac{BW_{\text{link}} \times \beta}{\text{oversubscription}} \quad (18)$$

This wall becomes binding when collective communication patterns demand bandwidth that the topology cannot supply at scale. For example, a 3D Torus cluster with 400 Gb/s links delivers an effective bisection bandwidth of $400 \times 0.67 = 268$ Gb/s per node; a Fat-Tree with the same links provides the full 400 Gb/s ($\beta = 1$), a $1.5\times$ advantage that compounds across multi-hop AllReduce patterns. We place the Locality Wall in the Data domain rather than Fleet because it models the *physical topology constraint* on data movement (bisection bandwidth, oversubscription), whereas the Communication Wall (Wall 14, Fleet) models the *algorithmic cost* of specific collectives (AllReduce, All-to-All) that run atop that topology. The two walls interact but address distinct levels of abstraction. **Assumption:** β values are topology-specific constants; real networks may exhibit dynamic congestion not captured by this static model.

4.3 Algorithm (Scaling & Compression)

The Algorithm walls arise not from hardware but from the mathematics of learning itself. They determine how much computation a workload *requires*, independent of the silicon that executes it.

Wall 11: The Complexity Wall. Chinchilla scaling laws (Hoffmann et al., 2022) establish that training compute scales jointly with model size P and dataset size D : doubling the parameters requires approximately doubling the tokens to remain compute-optimal. The `ScalingModel` implements:

$$C = 6PD \quad (\text{total training FLOPs}) \quad (19)$$

$$D^* \approx 20P \quad (\text{compute-optimal tokens}) \quad (20)$$

$$P^* = \sqrt{\frac{C}{120}} \quad (\text{optimal model size for budget } C) \quad (21)$$

These relations allow students to reason backward from a compute budget to the largest model that can be trained optimally, or forward from a model size to the minimum viable training cluster. For example, a budget of 10^{24} FLOPs yields $P^* \approx 91$ B parameters with $D^* \approx 1.8$ T tokens; Chinchilla (70B, 1.4T tokens) was trained near this optimal frontier (Hoffmann et al., 2022). **Assumption:** Scaling law coefficients are fitted to published training runs; extrapolation beyond the fitted range is flagged.

Wall 12: The Reasoning Wall. Inference-time compute scaling introduces a cost that grows linearly with the number of reasoning steps K . The `InferenceScalingModel` models this as (Snell et al., 2025):

$$T_{\text{reason}} = K \times T_{\text{step}}(P, S_{\text{context}}) \quad (22)$$

where T_{step} is the per-step latency, itself a function of model size P and context length S_{context} . In practice, K

varies dramatically by strategy: simple chain-of-thought uses $K \approx 3-8$, best-of- N sampling uses $K = N$ (typically 8–64), and tree-search methods like Monte Carlo Tree Search can require $K > 100$ (Snell et al., 2025). A $K=8$ chain-of-thought query costs $\sim 8\times$ more compute than a single-pass answer, fundamentally altering serving economics (see Case S1, Section 7). **Assumption:** Each reasoning step is an independent decode sequence; KV-cache is not shared across steps.

Wall 13: The Fidelity Wall. Compression trades model fidelity for efficiency. Quantization reduces precision while pruning removes weights entirely. The accuracy–efficiency frontier is task- and architecture-dependent. The `CompressionModel` quantifies the two primary mechanisms (Han et al., 2016; Gholami et al., 2021):

$$r_{\text{quant}} = \frac{b_{\text{base}}}{b_{\text{target}}} \quad (\text{quantization ratio, e.g., } 16/4 = 4\times) \quad (23)$$

$$r_{\text{prune}} = \frac{1}{1-s} \quad (\text{memory reduction at sparsity } s) \quad (24)$$

Here b_{base} is the baseline precision (typically 16 for FP16/BF16 models, not 32) and b_{target} is the quantized precision. Critically, quantization reduces memory reads but not FLOPs, shifting the arithmetic intensity rightward on the Roofline by a factor of r_{quant} and potentially crossing the ridge point from memory-bound to compute-bound. For pruning, r_{prune} gives the *memory reduction* ratio; actual compute speedup depends on sparsity structure. Unstructured sparsity yields no acceleration on current GPUs, while 2:4 structured sparsity (NVIDIA Corporation, 2023) provides a $2\times$ throughput gain via Sparse Tensor Cores. Post-training quantization methods such as GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024) demonstrate that 4-bit quantization can preserve most accuracy for large language models, making $r_{\text{quant}} = 4\times$ a practical operating point. Crucially, compression ratio alone does not determine inference speedup. Unstructured pruning reduces storage but yields no compute speedup on GPU GEMM kernels; only structured pruning and $N:M$ sparsity (e.g., 2:4 on NVIDIA Ampere+) accelerate hardware execution. The `CompressionModel` reports both `memory_savings_pct` and `inference_speedup` to capture this distinction. The accuracy impact Δ_{acc} is modeled as a configurable function, since the fidelity–compression frontier varies by architecture and task. **Assumptions:** Accuracy degradation follows empirical curves from Gholami et al. (2021); pruning compute speedup requires structured sparsity with hardware support.

4.4 Fleet (Multi-Node Coordination)

The Fleet walls arise when systems scale beyond a single node, requiring coordination across multiple accelerators connected by network fabric.

Wall 14: The Communication Wall. Distributed training requires gradient synchronization across N nodes, and the cost of that synchronization grows with both message size and node count. The `DistributedModel` models the dominant collective operations. For Ring AllReduce (Shoeybi et al., 2019) and its ZeRO/FSDP partitioned equivalents (Reduce-Scatter and All-Gather):

$$T_{\text{ring}} = \frac{2(N-1)}{N} \cdot \frac{M}{B_{\text{link}}} + 2(N-1) \cdot \alpha \quad (25)$$

where M is the message size, B_{link} is the per-link bandwidth, and α is the per-hop latency. Modern clusters use **hierarchical AllReduce** to exploit the bandwidth asymmetry between intra-node interconnect (e.g., NVLink at 900 GB/s) and inter-node fabric (e.g., InfiniBand at 50 GB/s per port, an $18\times$ gap). The `DistributedModel` implements a two-level model:

$$T_{\text{hier}} = T_{\text{intra}}(BW_{\text{NVLink}}, G) + T_{\text{inter}}(BW_{\text{IB}}, N) \quad (26)$$

where G is GPUs per node and N is the node count. Each level applies the ring formula (Equation (25)) at its respective bandwidth, making the inter-node phase the dominant cost at scale. After the intra-node reduce-scatter, each lead GPU holds M/G of the reduced result; the inter-node phase therefore operates on M/G , not the full M , making this the bandwidth-critical optimization of hierarchical collective design. In practice, modern frameworks also use **Compute/Communication Overlap**, hiding network latency behind backward pass computation. The `DistributedModel` models this with an overlap efficiency parameter $\eta_{\text{overlap}} \in [0, 1]$ (default 0.85, reflecting typical Megatron-LM behavior), yielding an exposed communication cost of $(1 - \eta_{\text{overlap}}) \cdot T_{\text{comm}}$. **Gradient Accumulation** further reduces effective communication cost. When accumulating gradients over K_{acc} microsteps before synchronizing, the DP AllReduce cost is amortized by $1/K_{\text{acc}}$, trading latency for communication efficiency at large global batch sizes. Tensor parallelism partitions model weights across devices and requires two AllReduce operations per transformer layer on *activations* (not weights). Each AllReduce exchanges a tensor of size $B \times S \times H \times b$ bytes, where B is batch size, S is sequence length, H is hidden dimension, and b is bytes per element. Weights are partitioned once and remain stationary. Pipeline parallelism (Narayanan et al., 2021) introduces a bubble overhead:

$$B_{\text{pipeline}} = \frac{P_{\text{stages}} - 1}{V \cdot M_{\text{micro}} + P_{\text{stages}} - 1} \quad (27)$$

where V is the number of virtual pipeline stages and M_{micro} is the number of microbatches. For Mixture-of-Experts All-to-All dispatch:

$$T_{\text{a2a}} = \frac{(N-1)}{N} \cdot \frac{M}{B_{\text{link}}} + (N-1) \cdot \alpha \quad (28)$$

where M is the routed activation payload. The `MoERoutingModel` keeps the MoE abstraction first-order: total parameters determine memory residency, active parameters determine compute, and hot-expert imbalance inflates the routed payload. With routing imbalance factor $\gamma \geq 1$:

$$E_{\text{active,eff}} = \min(E_{\text{total}}, \gamma \cdot E_{\text{top-k}}), \quad (29)$$

$$P_{\text{active,eff}} = P_{\text{active}} \cdot \frac{E_{\text{active,eff}}}{E_{\text{top-k}}}. \quad (30)$$

The same factor is available in `DistributedModel` as `moe_routing_imbalance_factor`, where it inflates expert-parallel traffic without introducing a router simulation. This follows the load-balancing motivation of sparsely-gated MoE and Switch-style routing systems (Shazeer et al., 2017; Fedus et al., 2022; Lepikhin et al., 2020).

Wall 15: The Fragility Wall. Component failures are inevitable at scale. If each node has a mean time between failures of $\text{MTBF}_{\text{node}}$, then a cluster of N nodes has (Daly, 2006):

$$\text{MTBF}_{\text{cluster}} = \frac{\text{MTBF}_{\text{node}}}{N} \quad (31)$$

The probability of at least one failure during a training run of duration T is:

$$P(\geq 1 \text{ failure}) = 1 - e^{-T/\text{MTBF}_{\text{cluster}}} \quad (32)$$

The Young-Daly formula (Young, 1974; Daly, 2006) gives the optimal checkpoint interval:

$$\tau_{\text{opt}} = \sqrt{2\delta \cdot \text{MTBF}_{\text{cluster}}} \quad (33)$$

where δ is the time to write one checkpoint. The `ReliabilityModel` uses these relations to estimate the fraction of compute lost to checkpointing and recovery, reporting a `goodput_ratio` that captures the effective fraction of wall-clock time spent on useful forward/backward computation after subtracting checkpoint writes and failure recovery. **Assumption:** Failures are independent and exponentially distributed (memoryless).

Wall 16: The Multi-tenant Wall. Production GPU clusters are rarely dedicated to a single job. Shared clusters introduce queueing delays that grow hyperbolically as utilization approaches 1.0, creating a tension between maximizing hardware utilization and minimizing researcher wait times. The `OrchestrationModel`

models job wait times using an M/D/1 queue (Little, 1961):

$$T_{\text{wait}} = \frac{\rho}{2\mu(1-\rho)} \quad (34)$$

where $\rho = \lambda/\mu$ is the cluster utilization, λ is the job arrival rate, and μ is the service rate. As $\rho \rightarrow 1$, wait times diverge hyperbolically. A cluster at 90% utilization has $5\times$ the queue depth of one at 50%. **Assumption:** Job durations are approximately deterministic, which is reasonable for large training runs with predictable step times.

4.5 Operations (Cost, Carbon & Safety)

The Operations walls capture constraints that are not about *how fast* a system runs but *whether it should run at all*: economic viability, environmental impact, checkpoint overhead, and responsible deployment.

Wall 17: The Capital Wall. Performance analysis is incomplete without economic constraints. A 1 024-GPU H100 cluster costs $\sim \$30.7\text{M}$ in CapEx alone; amortized over three years, the hardware cost of a single 30-day training run exceeds $\$800\text{K}$ before adding electricity. The `EconomicsModel` computes total cost of ownership (Barroso et al., 2018):

$$\text{TCO} = \text{CapEx} + \text{OpEx}_{\text{energy}} + \text{OpEx}_{\text{maint}} \quad (35)$$

where $\text{OpEx}_{\text{energy}} = E_{\text{total}} \times P_{\text{kWh}}$ converts total energy consumption to dollar cost at the regional electricity price. At scale, CapEx dominates. Electricity typically accounts for less than 10% of total run cost, making hardware utilization (MFU) the primary lever for cost efficiency. **Assumption:** Linear amortization over a 3–5 year hardware lifetime.

Wall 18: The Sustainability Wall. Regional grid carbon intensity varies by more than an order of magnitude, making geography a first-order systems design variable. The `SustainabilityModel` converts energy into environmental impact (Patterson et al., 2021):

$$E_{\text{total}} = E_{\text{IT}} \times \text{PUE} \quad (36)$$

$$\text{CO}_2 = E_{\text{total}} \times \text{CI}_{\text{region}} \quad (\text{gCO}_2/\text{kWh}) \quad (37)$$

$$\text{H}_2\text{O} = E_{\text{total}} \times \text{WUE} \quad (\text{L/kWh}) \quad (38)$$

where PUE is the power usage effectiveness of the datacenter, $\text{CI}_{\text{region}}$ is the carbon intensity of the local grid, and WUE is the water usage effectiveness. This model captures *operational* carbon only. For edge and IoT deployments at scale ($> 10^6$ devices), *embodied* carbon from manufacturing can dominate operational carbon by 10–100 $\times$ (Gupta et al., 2022). The `SustainabilityModel` accepts an optional `embodied_carbon_per_device` parameter for lifecycle analysis. **Assumption:** Grid

carbon intensity is a static regional constant; temporal variation (e.g., renewable intermittency) is not modeled. Energy-proportional power follows Barroso and Hölzle (2007): idle power is 30% of TDP, with the remaining 70% scaling linearly with MFU.

Wall 19: The Checkpoint Wall. Long-running training jobs must periodically save model state (weights and optimizer states) to persistent storage, incurring an I/O penalty that directly reduces effective MFU. The `CheckpointModel` models the I/O burst penalty (Eisenman et al., 2022):

$$\text{MFU}_{\text{penalty}} = \frac{T_{\text{write}}}{T_{\text{interval}}} = \frac{|W| \times \beta_{\text{opt}} / BW_{\text{storage}}}{T_{\text{ckpt_interval}}} \quad (39)$$

where $|W|$ is the model weight size in bytes, and β_{opt} is the optimizer state multiplier, the ratio of total checkpoint bytes to model weight bytes. For mixed-precision Adam with FP16 weights, $\beta_{\text{opt}} \approx 7$ (FP32 master weights at 4 bytes + FP32 momentum at 4 bytes + FP32 variance at 4 bytes, plus FP16 model weights at 2 bytes, totaling 14 bytes per parameter vs. 2 bytes for the FP16 model alone). Gradients are ephemeral and not checkpointed. For a 70B-parameter model, the checkpoint size is $70\text{B} \times 14 \text{ B/param} \approx 1 \text{ TB}$, making storage bandwidth the binding constraint during I/O bursts.

Wall 20: The Safety Wall. Privacy and fairness guarantees impose quantifiable computational overhead. The `ResponsibleEngineeringModel` models the cost of differential privacy via DP-SGD (Abadi et al., 2016), where the noise multiplier scales inversely with the privacy budget:

$$\sigma \propto \frac{1}{\epsilon} \quad (40)$$

At moderate privacy ($\epsilon = 1$), DP-SGD typically incurs a $\sim 3\times$ training slowdown due to per-sample gradient clipping and noise addition; at strong privacy ($\epsilon = 0.1$), the overhead can exceed $10\times$. The solver models this as a compute multiplier: $T_{\text{DP}} = T_{\text{base}} \times f(\epsilon)$, where $f(\epsilon)$ is empirically calibrated from published DP-SGD benchmarks. Fairness constraints impose a complementary data requirement. Sufficient representation of minority subgroups demands additional training data proportional to $O(1/p_{\text{min}})$, where p_{min} is the smallest subgroup prevalence. Unlike the other walls, Wall 20 is more qualitative than quantitative, with overhead depending heavily on task, model architecture, and privacy mechanism. The solver reports the compute multiplier as a range rather than a point estimate. **Assumption:** Privacy budget ϵ is a hard constraint; the solver reports the compute multiplier, not the privacy guarantee.

4.6 Analysis (Cross-Cutting Diagnostics)

The preceding 20 walls each model a specific physical or logical constraint. The final two entries are *diagnostic tools* rather than walls in the strict sense. They operate *across* the taxonomy rather than within a single domain, providing analysis capabilities that span all walls. The Sensitivity tool identifies which wall is binding, and the Synthesis tool derives minimum hardware from SLA requirements.

Wall 21: The Sensitivity Wall. Optimization is effective only when directed at the binding constraint; improving a non-bottleneck parameter yields no measurable gain. The `SensitivitySolver` identifies the binding constraint by computing partial derivatives of end-to-end latency with respect to each hardware parameter (Williams et al., 2009):

$$\frac{\partial T}{\partial BW_{\text{mem}}}, \quad \frac{\partial T}{\partial \text{Peak}_{\text{FLOPS}}}, \quad \frac{\partial T}{\partial BW_{\text{net}}}, \quad \dots \quad (41)$$

The parameter with the largest sensitivity is the binding constraint, that is, the single upgrade that would yield the greatest performance improvement. Each reported sensitivity is the normalized latency response $(T_{\text{perturbed}} - T_{\text{base}}) / T_{\text{base}}$ to a 10% parameter upgrade. For Llama-3 8B decode on an A100, the solver reports a memory-bandwidth sensitivity of -0.088 (a 10% bandwidth upgrade cuts latency by 8.8%) versus exactly zero for peak FLOPS, because under the hard max of Equation (3) a strictly memory-bound workload does not respond to a compute upgrade at all. This transforms “where should I invest?” from intuition into calculation. **Assumption:** Finite-difference approximation with 10% perturbation; second-order effects are ignored. The binding constraint is identified as the parameter with the largest absolute sensitivity.

Wall 22: The Synthesis Wall. The `SynthesisSolver` addresses the inverse problem: given a service-level objective (e.g., 50 ms inter-token latency), it derives the minimum hardware specifications required to satisfy it (Kwon et al., 2023):

$$BW_{\text{required}} = \frac{|W|}{T_{\text{target}}} \quad (42)$$

$$\text{FLOPS}_{\text{required}} = \frac{\text{OPs}}{T_{\text{target}} \times \eta} \quad (43)$$

This enables hardware-software co-design. Engineers specify an SLA and the solver derives the minimum hardware that satisfies it. For LLaMA-70B at a 50 ms inter-token latency target, the solver yields $BW_{\text{required}} = 140 \text{ GB}/50 \text{ ms} = 2800 \text{ GB/s}$, $1.4\times$ the A100’s 2 TB/s, confirming that this SLA requires at least two tensor-parallel A100s. **Assumption:** Hardware parameters are

independently adjustable; co-design coupling between FLOPS and bandwidth is not modeled.

5 The 3-Tier Resolver Architecture

The walls define *what* constrains a system. This section formalizes *how* resolvers compose to produce end-to-end system evaluations. To clarify the mathematical intent of each component, MLSYS·IM organizes its analytical tools into a strict 3-tier taxonomy, summarized below before Section 5.1 shows how the tiers chain into multi-stage analyses through a shared, dimensionally typed contract, and Section 5.2 describes the three-level `SystemEvaluation` scorecard (Feasibility, Performance, Macro) that the framework returns.

Tier 1: Analytical Models (the “physics engine”). Models perform forward evaluation $Y = f(X)$ to determine the physical and logical consequences of a specific system configuration. For example, the `ServingModel` calculates the exact time-to-first-token for a given LLM and GPU pair. Models are purely deterministic and make no decisions; they comprise the first 23 resolvers in our taxonomy.

Tier 2: Analysis Solvers (the “math engine”). Solvers perform algebraic inversion or calculus, $X = f^{-1}(Y)$ or ∇f , to find the exact parameter required to hit a specific target. For example, the `SynthesisSolver` takes a target latency SLA and works backward to derive the minimum memory bandwidth required.

Tier 3: Optimizers (the “engineering engine”). Optimizers perform constrained design-space search, $\max f(X)$ subject to $g(X) \leq c$, to find the best configuration among many valid options. Unlike Models and Solvers, which map directly to individual walls, Optimizers operate across the entire taxonomy. The `ParallelismOptimizer` sweeps all valid 3D tensor/pipeline/data parallel splits to maximize Model FLOPs Utilization (MFU) on a given cluster; the `BatchingOptimizer` searches for the maximum batch size that satisfies a P99 queueing latency SLA.

5.1 Stateless Composition and Chaining

Every resolver in MLSYS·IM is a pure function: it accepts a typed configuration, performs analytical computation, and returns a typed result object (a `Pydantic BaseModel` with dimensioned fields). Resolvers maintain no hidden state between invocations. Because they share a common type system, the output of one tier feeds naturally into the next, as illustrated in Figure 5. The reader should trace, in that figure, how the demand,

supply, and topology layers feed a resolver chain that culminates in the three-level scorecard described in Section 5.2; the typed contract at each arrow is what enables the chain to compose without runtime checks. A full-stack analysis composes resolvers in sequence to resolve complex design questions. For example, determining the financial cost of training an optimally-sized model on a frontier cluster requires chaining algorithmic scaling, distributed execution, and macro-economics:

$$\text{Scaling} \xrightarrow{\mathcal{R}_1} \text{Distributed} \xrightarrow{\mathcal{R}_2} \text{Economics} \xrightarrow{\mathcal{R}_3} \text{Sustainability} \quad (44)$$

Listing 3 demonstrates this exact chain in MLSYS·IM. The `ScalingModel` calculates the optimal model size for a given compute budget ($\mathcal{R}_1$). The `DistributedModel` takes that workload and computes the real-world execution time on an 8,192-GPU fleet, factoring in 3D parallelism overhead ($\mathcal{R}_2$). Finally, the `EconomicsModel` converts that execution time into a Total Cost of Ownership ($\mathcal{R}_3$).

Each link in the chain preserves dimensional correctness. Units propagate through the computation, and any mismatch raises an immediate error rather than producing a silently wrong result.

5.2 The SystemEvaluation Scorecard

MLSYS·IM provides a `Scenario.evaluate()` entry point that orchestrates solver composition automatically through a three-level evaluation. A scenario is a concrete executable case, such as a doorbell workload on a microcontroller or a frontier training workload on a fleet; sourced statistics used only as reference anchors remain outside the executable path in `ReferenceStats.*`.

Level 1: Feasibility. Does the model fit? Can the data pipeline keep pace? The framework checks memory capacity against model size, ingestion bandwidth against training throughput, and reports any wall where demand exceeds supply.

Level 2: Performance. What are the achievable latency, throughput, and utilization? The Roofline analysis (Equation (3)), communication modeling (Equation (25)), and pipeline bubble (Equation (27)) combine to produce end-to-end training step time.

Level 3: Macro. What does it cost, and what does it emit? TCO (Equation (35)), carbon (Equation (36)), and responsibility overhead (Equation (40)) are computed from the performance results.

The three levels are evaluated in order; a feasibility failure at Level 1 short-circuits the evaluation and reports the binding constraint. This ordering reflects the dependency structure, since communication optimization is irrelevant if the model exceeds available memory. The complete implementation details and key


```

1 import mlsysim
2 from mlsysim.solvers import ScalingModel,
  DistributedModel, EconomicsModel
3 from mlsysim.models.types import
  TransformerWorkload
4
5 # 1. Algorithm: Find optimal parameters for a fixed
  compute budget
6 budget = mlsysim.Q_("4e24 flop") # ~100K H100-days
  at 50% MFU
7 optimal = ScalingModel().solve(compute_budget=
  budget) # P* ~ 183B params
8
9 # Instantiate the demand (Layer A: Workload)
10 model = TransformerWorkload(
11     name="Frontier-Model", architecture="
  transformer",
12     parameters=optimal.optimal_parameters,
13     layers=80, hidden_dim=8192, heads=64
14 )
15
16 # 2. Fleet: Evaluate on a massive 8K GPU cluster (
  Layer D: Supply/Topology)
17 fleet = mlsysim.Systems.Clusters.Frontier_8K
18 seq_len = 4096
19 perf = DistributedModel().solve(
20     model, fleet,
21     batch_size=4096, tp_size=8, pp_size=4, seq_len=
  seq_len
22 )
23
24 # 3. The Capital: Calculate TCO for the resulting
  training time
25 n_steps = optimal.optimal_tokens.to("count").
  magnitude / (4096 * seq_len)
26 duration = perf.step_latency_total * n_steps
27 tco = EconomicsModel().solve(fleet, duration_days=
  duration.to("day").magnitude)
28
29 print(f"Scaling Efficiency: {perf.
  scaling_efficiency:.1%}") # 54.9%
30 print(f"Training Duration: {duration.to('day'):.1f
  }") # 68.1 day
31 print(f"Total Job Cost: ${tco.tco_usd:,.0f}")
  # $15,905,569

```

Listing 3: **Solver Composition.** Bridging algorithmic scaling, distributed execution, and fleet economics in a single executable chain.

assumptions for each solver are documented alongside their respective walls in Section 4.

6 Validation

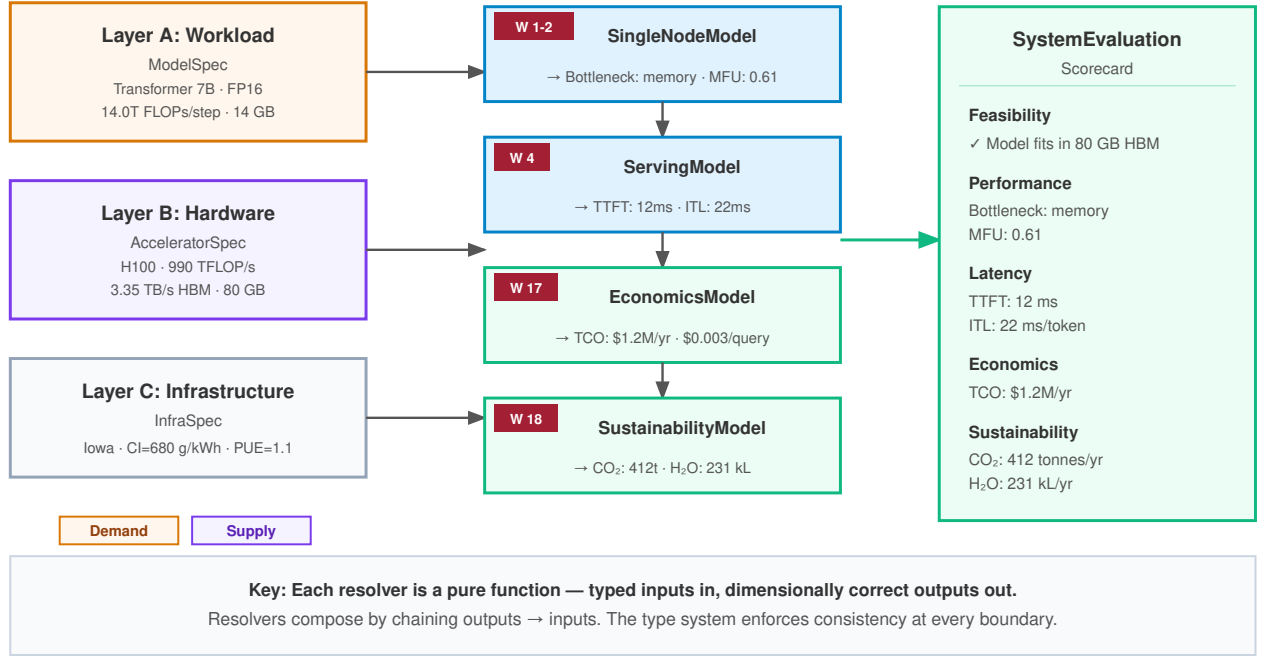
An analytical framework earns trust through transparent confrontation with empirical ground truth. We validate MLSYS·IM along two axes: *accuracy* against published benchmarks, and *speed* relative to alternative modeling tools. Crucially, our goal in validation is not to “curve-fit” the model to perfectly match empirical results by introducing arbitrary fudge factors. Rather, it is to demonstrate that a first-principles application of physics and generalized efficiency constants places the prediction squarely in the correct ballpark. As we argue in Section 9, a model that is off by 10% because it structurally ignores L2 cache misses is functioning exactly as intended, trading cycle-level precision for architectural intuition. The anchors below illustrate this philosophy.

The remainder of this section presents the validation in four parts. Section 6.1 anchors MLSYS·IM predictions against seven published benchmarks spanning single-node training, distributed training, inference, scaling laws, sustainability, and automated parallelism search. Section 6.2 reports the design-space exploration speed relative to cycle-accurate simulators. Section 6.3 discusses the accuracy scope and the empirical role of the efficiency parameter η . Finally, Section 6.4 argues that runtime dimensional strictness is itself a structural form of validation that numerical accuracy alone cannot supply.

6.1 Empirical Anchors

We anchor MLSYS·IM predictions against seven published benchmarks spanning single-node training, distributed training, inference, scaling laws, sustainability, and automated design-space optimization.

Anchor 1: MLPerf ResNet-50 on DGX A100 (Single-Node Training). For ResNet-50 training on a DGX A100 node (8× A100 GPUs with NVLink) at batch size 2048, MLSYS·IM predicts a throughput of approximately 37,500 samples/s using the `SingleNodeModel` with end-to-end hardware utilization $\eta = 0.19$ and 8-way data parallelism within the node. The low η reflects that ResNet-50’s small convolution kernels, data pipeline, and framework overhead cannot saturate the A100’s tensor cores the way large GEMM-dominated Transformer layers can. NVIDIA’s MLPerf Training closed-division submissions for the 8-GPU DGX A100 configuration (Mattson et al., 2020) report aggregate training rates in the same regime as our prediction; at the order of magnitude that determines whether the system is compute-bound or memory-bound, this validates the Roofline-based throughput model (Williams et al., 2009) at the core of MLSYS·IM’s single-node solver. Per-



Resolver chaining enables full-stack analysis: from silicon constraints through economics to environmental impact.

Figure 5: **Resolver Composition.** Three input layers feed four resolvers. Each resolver is a pure function: typed inputs in, dimensionally correct outputs out. The scorecard aggregates three evaluation levels: Feasibility, Performance, and Macro (economics, sustainability, safety).

GPU throughput is $\sim 4,700$ samples/s, consistent with the A100’s Roofline ceiling for ResNet-50’s arithmetic intensity.

Anchor 2: vLLM Llama-2-70B on H100 (Inference).

For autoregressive decoding of Llama-2-70B (FP16, batch size 1), the model weights total 140 GB, requiring at minimum two tensor-parallel H100s (each with 80 GB HBM3). MLSYS · IM’s first-order estimate divides total weights by aggregate bandwidth: $140 \text{ GB} / (2 \times 3.35 \text{ TB/s}) = 20.9 \text{ ms}$ for the weight-read phase alone. Adding KV-cache reads, attention computation, NVLink synchronization, and framework scheduling overhead, the predicted end-to-end ITL is approximately 43 ms (NVIDIA Corporation, 2023). Published vLLM benchmarks for this configuration report ITL values in the 40–50 ms range (Kwon et al., 2023), confirming that decode-phase LLM inference is memory-bandwidth-bound and that the overhead multiplier ($\sim 2\times$ over the pure bandwidth floor) is consistent across deployments.

Anchor 3: Llama 3 Training at 16K H100s (Distributed Training). Meta’s Llama 3 training report (Llama Team, AI Meta, 2024) documents achieving 38–43% MFU on 16,384 H100 GPUs with 4D parallelism ($\text{TP} \times \text{CP} \times \text{PP} \times \text{DP}$). We configure MLSYS · IM’s DistributedModel with the published fleet (2,048 nodes $\times$ 8 H100s, 400 Gb/s InfiniBand per

node) training the 405B-parameter model with intra-node tensor parallelism ($\text{TP}=8$), pipeline parallelism over 64 microbatches, and system-level efficiency $\eta = 0.42$. This η captures kernel utilization, stragglers, load imbalance, checkpointing pauses, and thermal throttling, effects that the analytical communication model does not represent. After accounting for pipeline bubble overhead (Equation (27)), hierarchical AllReduce cost (Equation (26)), and compute–communication overlap ($\eta_{\text{overlap}} = 0.85$), MLSYS · IM predicts a scaling efficiency of $\sim 93\%$, yielding an aggregate MFU of $0.93 \times 0.42 \approx 39.1\%$, within the reported 38–43% range. This validates the distributed training model at production scale.

Anchor 4: PaLM Hardware vs. Model FLOPs Utilization (Communication Overhead). Google’s PaLM-540B (Chowdhery et al., 2023) was trained on 6,144 TPU v4 chips (two pods of 3,072 connected over the data-center network), achieving 57.8% hardware FLOPs utilization but only 46.2% model FLOPs utilization (MFU). The gap reflects activation rematerialization and the cross-pod gradient reduction that the raw hardware-utilization figure hides. Using MLSYS · IM’s DistributedModel with TPU v4 specifications (275 TFLOP/s BF16, ICI bandwidth at 24 GB/s with $2\times$ oversubscription) and the PaLM-540B workload at

system-level efficiency $\eta = 0.47$, the predicted aggregate MFU is $\sim 43\%$, within ~ 3 percentage points of the reported 46.2%. The η absorbs ICI fabric congestion, straggler effects, and scheduling overhead. `MLSYS·IM` correctly identifies the intra-pod to inter-pod bandwidth transition, the cross-pod data-parallel gradient exchange over the slower data-center network, as the dominant communication bottleneck.

Anchor 5: Chinchilla Scaling Laws (Algorithmic Scaling). The Chinchilla paper (Hoffmann et al., 2022) establishes that compute-optimal training requires $D \approx 20P$ tokens. `MLSYS·IM`’s `ScalingModel` implements the parametric scaling law $C = 6PD$ and derives the optimal allocation $P^* = \sqrt{C/120}$. For $C = 10^{24}$ FLOPs, the solver predicts $P^* \approx 91B$ parameters with $D^* \approx 1.8T$ tokens. Chinchilla itself was trained at $C = 6 \times 70B \times 1.4T \approx 5.88 \times 10^{23}$ FLOPs, for which the solver predicts $P^* \approx 70.0B$, recovering the published 70B model size to better than 1%. This validates the scaling law implementation against its original calibration data.

Anchor 6: Training Carbon Footprint (Sustainability). Patterson et al. (2021) report that training GPT-3 (175B parameters) on V100 GPUs consumed approximately 1,287 MWh and emitted 552 tonnes CO₂. Using the reported energy and US average grid intensity (429 gCO₂/kWh), `MLSYS·IM`’s carbon-accounting layer computes $1,287 \times 429 / 1,000 = 552$ tonnes CO₂, matching the reported footprint to rounding. This validates the dimensional carbon calculation separately from any hardware power model.

Anchor 7: Llama 3 Parallelism Strategy (Optimizer Convergence). To validate the Tier 3 design-space search, we configure the `ParallelismOptimizer` with the Meta Llama 3 405B model and its 16,384 H100 cluster constraints (Llama Team, AI Meta, 2024). The optimizer sweeps power-of-two TP and PP degrees with $TP \times PP \times DP = 16,384$, pre-screening each candidate so that the per-GPU shard of weights, gradients, and Adam optimizer state (the dominant training-memory term) fits in 90% of the 80 GB HBM capacity, then ranking survivors by MFU. It recovers Meta’s binding decisions on both parallelism axes: TP=8, pinning tensor parallelism to a single node so AllReduce traffic stays on intra-node NVLink rather than crossing InfiniBand, and PP=16, the shallowest pipeline whose per-GPU memory screen passes once optimizer state is counted. This matches the TP=8, PP=16 parallelism backbone Meta reports; the optimizer allocates the remaining degrees to data parallelism (DP=128), whereas Meta additionally folds in context parallelism (CP=16) for batch-size and activation-memory reasons beyond the optimizer’s weight-and-optimizer-state screen.

These seven anchors span five of the six taxonomy domains (Node, Data is validated indirectly via the ResNet pipeline-bound case in Section 7, Algorithm, Fleet, Operations) and cover both Roofline regimes (compute-bound and memory-bound). Table 6 summarizes the results.

Every entry across all eight MLSys Zoo registries records mandatory `metadata.provenance` (Section 3.1.1). This enforces strict traceability: datasheet, literature, and industry-report numbers require a verified URL and date, while estimates and derived values require documented justification, and the provenance audit tool (`mlsysim.tools.audit_provenance`) fails on any registry value whose provenance is missing or weak. This prevents the silent accumulation of “magic numbers” as the framework evolves.

6.2 Design-Space Exploration Speed

`MLSYS·IM`’s analytical engine sweeps over 1,000 hardware–model–precision configurations in under one second on a standard laptop. In contrast, ASTRA-sim 2.0 requires hours to simulate a single distributed training configuration at cycle-level fidelity (Won et al., 2023). This three-order-of-magnitude speedup is the design objective. Sub-second execution enables interactive parametric sweeps (e.g., varying HBM bandwidth, substituting fat-tree for torus topology, or relocating the datacenter from Iowa to Singapore) that would be impractical with cycle-accurate simulation. Figure 6 illustrates one such sweep, a 42-point grid of model size versus HBM bandwidth, where each cell represents a single solver invocation and the entire map executes in under 50 ms on a standard laptop.

6.3 Accuracy Scope and Limitations

`MLSYS·IM` provides first-order estimates, not cycle-accurate predictions. We do not re-derive the resolver abstraction here. It is the same first-principles structural bounds plus a single efficiency η for unmodeled second-order effects, as in Sections 1 and 3.1. **Calibrating η for validation.** Crucially, η must be selected for the workload class and measurement scope: $\eta \approx 0.19$ for end-to-end ResNet-50 training throughput on A100, reflecting convolution, framework, and data-pipeline overhead (Mattson et al., 2020); $\eta \approx 0.42$ for large-scale Transformer training, consistent with Meta’s reported system-level efficiency (Llama Team, AI Meta, 2024). The Roofline parallel (peak vs. achievable ceilings; which constraint binds) is already stated in Section 1; see Williams et al. (2009). Two of the seven anchors above require no efficiency parameter at all, because the Chinchilla scaling law is pure mathematics and the carbon footprint is a direct measurement, demonstrating

Table 6: **Validation Summary.** Predicted vs. reported values across seven empirical anchors. Error is $|(pred. - rep.)/rep.|$.

Anchor	Predicted	Reported	Error
1: ResNet-50 DGX A100	37,500 s/s	v0.7 same order	order-of-mag.
2: Llama-2 70B ITL	43 ms	40–50 ms	in range
3: Llama 3 MFU	39.1%	38–43%	in range
4: PaLM MFU	43% MFU	46.2% MFU	6.9%
5: Chinchilla P^*	70.0B	70B	<1%
6: GPT-3 CO ₂	552 t	552 t	0.0%
7: Llama 3 Parallelism	TP=8, PP=16, DP=128	TP=8, PP=16, CP=16	TP, PP exact

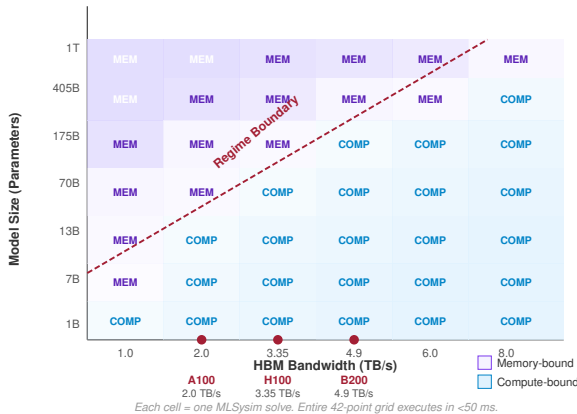


Figure 6: **Design-Space Exploration: Bottleneck Regime Map.** Each cell shows the binding constraint (memory or compute) for a given model size and HBM bandwidth combination under FP16 training at batch size 256. Larger models with lower bandwidth are memory-bound; smaller models with higher bandwidth are compute-bound. The diagonal regime boundary shifts as hardware generations increase bandwidth (A100 → H100 → B200). The entire 42-point grid executes in <50 ms.

that the physics layer is independently sound. Section 9 details the specific phenomena this abstraction cannot capture and the resulting accuracy boundaries.

For MLSYS·IM’s intended use cases (architectural reasoning, lab exercises, capacity planning, and design-space exploration), first-order accuracy is sufficient and often preferable. A student who understands *why* a system is memory-bound has learned more than one who can predict its throughput to three decimal places.

To safeguard against model drift, the test suite includes empirical anchor tests (`engine.empirical`) that fail automatically if predictions leave their reviewed tolerance envelopes around published values.

6.4 Dimensional Correctness as Validation

Beyond numerical accuracy, dimensional strictness (Section 3.1.2) provides a structural form of validation. Unit mismatches raise immediate errors rather than producing silently wrong results, eliminating a category of bugs that numerical validation alone cannot catch.

6.5 Design Invariant Enforcement

Dimensional strictness (Section 3.1.2) catches unit errors at runtime. A complementary question is whether the framework’s *architectural* invariants hold as the codebase evolves: do all hardware specifications flow through typed registries? Does every formula carry `pint` units end-to-end? Can a contributor accidentally reintroduce a bare constant that bypasses the registry? MLSYS·IM answers these with five machine-checked structural invariants, enforced by an 862-test suite, each encoding a design decision that would otherwise erode through incremental changes:

1. **Specification separation.** There is no constants module at all: the legacy `core/constants.py` was deleted outright once its last value migrated, and a CI gate now pins its *absence* (any attempt to reintroduce the module, or a compatibility shim for it, fails), alongside name-pattern checks for hardware, model, and network constants. Every domain figure lives in its category registry; measurement units live in `core.units`.
2. **Loader contract.** The YAML registry loader rejects duplicate keys outright and requires every `@tech:` and `@prov:` reference to resolve, so a typo in a data file fails at import rather than silently shadowing or orphaning a specification.
3. **Provenance coverage.** A dedicated audit (`mlsysim.tools.audit_provenance`, also run as a test) walks every registry leaf and fails on missing or weak provenance, enforcing the kind-specific evidence rules of Section 3.1.1 across all eight zoos.

4. **Cross-registry consistency.** When the same physical quantity appears in multiple registries (e.g., NVLink bandwidth in both **Hardware** and **Systems** node definitions), a dedicated check verifies that the values agree. This prevents silent divergence between overlapping specification surfaces.
5. **Registry immutability.** Every registry entry is a frozen `pydantic` model: attribute assignment raises at the call site. Entries are shared singletons read by every notebook cell in a rendering process, so without this guarantee a single stray write (`H100.tdp = ...`) would silently corrupt every downstream calculation; with it, the mistake fails loudly where it happens. Code that needs a perturbed variant builds an explicit copy (`model_copy`).

These checks are additive: each new registry entry or physics module automatically participates without manual test authoring. Empirical-anchor tests (`engine.empirical`) complete the picture by binding representative solver outputs to sourced benchmark envelopes, so a formula regression or a registry edit that moves a prediction outside its reviewed range fails CI. The effect is that the zoo architecture described in Section 3.2 is not merely documented but *enforced* — a structural guarantee that the framework’s design decisions survive community contributions and long-term evolution.

7 Usage & Case Studies

MLSYS·IM is designed for three audiences: students developing quantitative reasoning skills, instructors preparing demonstrations, and researchers evaluating design trade-offs. We present two representative use cases per persona, each illustrating how solvers compose to answer questions that span multiple walls. For a "Cookbook" of exciting, runnable code examples demonstrating how users define hardware, perform algebraic design-space sweeps, and invoke SLA-driven hardware synthesis programmatically in Python, see the Appendix (Listings 5, 6, 7, 8, 9, 10, 11, 12, and 13).

7.1 Student Use Cases

Students interact with MLSYS·IM primarily through single-solver queries, short resolver chains, and browser-friendly exercises in the companion textbook. Students can manipulate hardware parameters (e.g., batch size, SLA targets, carbon taxes) and observe how binding constraints shift without needing a backend server or physical hardware.

We present two examples chosen to illustrate complementary aspects of the framework. The first (S1) demonstrates *vertical* resolver composition, chaining inference

and economics models to connect an algorithmic decision (chain-of-thought reasoning) to its infrastructure cost. The second (S2) demonstrates *horizontal* composition, combining data-pipeline and compute models to diagnose a bottleneck that shifts between walls as batch size increases.

7.1.1 S1: Chain-of-Thought Cost

A student investigates the inference economics of chain-of-thought (CoT) prompting. Using Llama-3 70B on H100 hardware, they configure the `InferenceScalingModel` with $K=8$ reasoning steps. Each step generates ~ 50 tokens (the calibrated default) at the memory-bound decode rate, so the total reasoning time is $T_{\text{reason}} = \text{TTFT} + K \cdot 50 \cdot \text{ITL}$. The solver reports a total reasoning time of ~ 17.9 s, a $6.5\times$ per-query *latency* multiplier relative to a single-step answer (the one-time prefill TTFT amortizes across the K steps). The *cost* multiplier is larger and tracks total work rather than latency: CoT generates $8\times$ as many decode tokens ($K \cdot 50$ vs. 50), and because decode is memory-bandwidth-bound the serving throughput is set by token volume. Feeding this into the `EconomicsModel`, a deployment sized for 100 QPS of single-pass queries must grow its H100 fleet, and hence its TCO, by $\sim 8\times$ to sustain the same query rate under CoT. The two multipliers are distinct: MLSYS·IM shows chain-of-thought taxes both latency and infrastructure cost, but by different factors.

7.1.2 S2: CPU Pipeline Bottleneck

A student configures ResNet-50 training on a DGX A100 (8 GPUs) with a batch size of 2,048. At a kernel-level efficiency of $\eta = 0.30$, the `SingleNodeModel` predicts a per-step compute time of ~ 35 ms, a demand rate of $\sim 58,000$ images/s. Adding the `DataModel` rules out storage I/O. With 64 CPU workers (8 per GPU) decoding ImageNet JPEGs at 1,200 images/s each, the raw decode pipeline delivers 76,800 images/s. This appears sufficient, but the `TransformationModel` accounts for the full augmentation pipeline (random crop, color jitter, normalization) at 850 images/s per worker, reducing effective throughput to $64 \times 850 = 54,400$ images/s, below the GPUs’ 58,000 images/s appetite. The student discovers that the binding constraint shifts from Wall 1 (Compute) to Wall 9 (Transformation): the GPUs have spare cycles, but the CPUs cannot feed them fast enough, and the achieved *end-to-end* efficiency degrades below the kernel-level η , consistent with the gap between kernel-level and end-to-end efficiency in Anchor 1 ($\eta = 0.19$, Section 6.1).

7.2 Instructor Use Cases

Instructors need demonstrations that run in real time, produce concrete numbers, and connect cleanly to lecture narratives. The following cases show how MLSYS·IM turns abstract concepts into live, interactive classroom demonstrations.

7.2.1 I1: Live Roofline Demo, Batch Size Sweep

An instructor demonstrates the Roofline model by sweeping batch size from 1 to 256 on an H100 for ResNet-50 inference at $\eta = 0.5$. At each batch size, the `SingleNodeModel` returns the bottleneck label, the arithmetic intensity, and the MFU:

Batch	Bottleneck	AI (FLOP/B)	MFU
1	Memory	73	0.01
8	Compute	356	0.06
32	Compute	610	0.17
128	Compute	743	0.34
256	Compute	771	0.40

The crossover from memory-bound to compute-bound occurs between batch 1 and batch 8, where the arithmetic intensity crosses the effective ridge point $\eta \times F_{\text{peak}}/BW_{\text{HBM}} \approx 148 \text{ FLOP/byte}$: batching amortizes the one-time weight read across samples, while per-sample activation traffic bounds the gain. MFU keeps climbing past the crossover ($0.06 \rightarrow 0.40$) as the fixed dispatch and per-layer framework taxes amortize. Figure 7 visualizes this transition on the Roofline diagram. The contrast case makes the deeper point: repeating the sweep for a 7B-parameter LLM *decode* workload shows the arithmetic intensity saturating near 10 FLOP/byte, far left of the ridge, because KV-cache and activation reads grow with the batch; decode remains memory-bound at every batch size (Walls 4–5). The entire sweep executes in <50 ms, enabling real-time interaction during lecture.

7.2.2 I2: Iowa vs. Québec Carbon

An instructor poses a policy question: *does geography matter for carbon footprint?* Using the `DistributedModel`, they configure a 256-GPU cluster (`Systems.Clusters.Research_256`) training a 70B model for 30 days at MFU 0.42. The `SustainabilityModel` then computes emissions under two registered datacenter profiles: Iowa (680 gCO₂/kWh, PUE 1.12, circa 2020 coal/gas grid) and Québec (20 gCO₂/kWh, PUE 1.06, hydroelectric). The configuration is identical in hardware, model, and MFU, yet the run emits 58.4 tonnes CO₂ in Iowa versus 1.6 tonnes in Québec, a 36× gap, demonstrating that grid carbon intensity is a first-order systems design variable (Wall 18). Figure 8 visualizes this contrast.

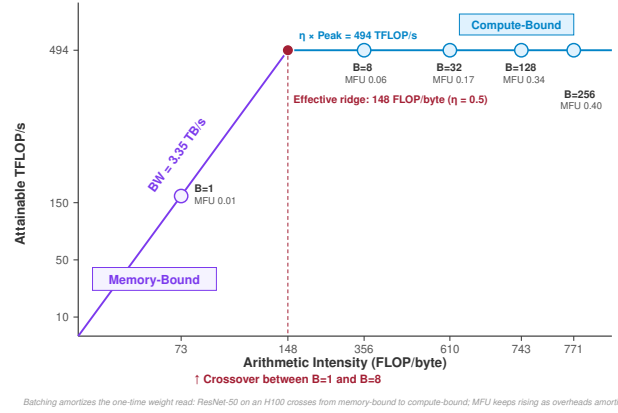


Figure 7: **Roofline Crossover: ResNet-50 Batch Size Sweep on H100.** Increasing batch size moves the operating point rightward along the Roofline, transitioning from memory-bound (purple) to compute-bound (cyan) between batch 1 and batch 8. The peak ridge point is $989/3.35 \approx 295 \text{ FLOP/byte}$; at $\eta = 0.5$ the effective ridge sits at $\approx 148 \text{ FLOP/byte}$. MFU continues rising past the crossover as dispatch and framework overheads amortize.

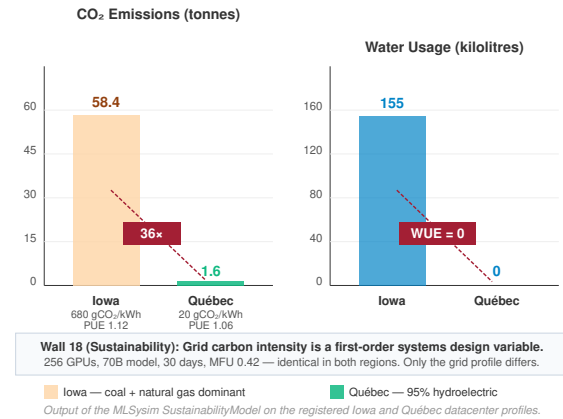


Figure 8: **Geography as a Systems Variable: Iowa vs. Québec.** An identical 256-GPU cluster training a 70B model for 30 days produces 36× more CO₂ in Iowa (coal/gas grid circa 2020, 680 gCO₂/kWh, PUE 1.12) than in Québec (95% hydroelectric, 20 gCO₂/kWh, PUE 1.06): 58.4 vs. 1.6 tonnes. Water usage diverges even more sharply because Québec’s registered WUE is zero. Hardware, model, parallelism strategy, and MFU are identical; only the regional grid profile differs between the two sites.

7.3 Researcher Use Cases

Researchers need to evaluate architectural alternatives and justify procurement decisions with quantitative evidence. The following cases show how MLSYS·IM en-

ables rapid what-if analysis across hardware generations and parallelism strategies.

7.3.1 R1: GPU vs. Cerebras Crossover

A researcher evaluates whether wafer-scale silicon changes the economics of large-model inference. For a 180B-parameter model on H100s (the 360 GB FP16 checkpoint requires multi-GPU tensor parallelism across 5 GPUs), `MLSYS·IM` reports a decode-phase ITL of $360 \text{ GB} / (5 \times 3.35 \text{ TB/s}) = 21.5 \text{ ms/token}$ at batch 1 (memory-bound). The `WeightStreamingModel` then models the Cerebras WSE-3, assuming a MemoryX injection bandwidth of 1.2 TB/s (Cerebras has not published an official MemoryX bandwidth figure, so we treat this as a calibration estimate). At batch size 1, the full 360 GB must stream from MemoryX at this rate, yielding a per-token latency of $360 / 1,200 = 300 \text{ ms}$, $14\times$ slower than the GPU cluster. However, the Cerebras architecture’s advantage emerges at scale. At the optimal batch $B^* \approx 41,700$ (Equation (14), at $\eta = 0.4$), injection cost is fully amortized across tokens, achieving $\sim 139,000$ tokens/s aggregate throughput versus ~ 47 tokens/s for the five-GPU tensor-parallel replica (requiring a fleet of $\sim 15,000$ H100s to match). The solver flags that at B^* , the activation footprint approaches the 44 GB SRAM ceiling. The `SensitivitySolver` confirms the qualitative regime change. The binding constraint shifts from BW_{HBM} (GPU) to $\text{BW}_{\text{inject}}$ (WSE-3), illustrating that the optimal architecture depends critically on batch size.

7.3.2 R2: Hardware Procurement Audit

A researcher preparing a hardware procurement recommendation for LLM inference needs to answer: *should the next-generation cluster prioritize FLOPS or bandwidth?* The `SensitivitySolver` perturbs each hardware parameter by 10% and reports the normalized latency response, shown here for a single-GPU-resident Llama-3 8B decode workload:

```
1 import mlsysim
2 from mlsysim.solvers import SensitivitySolver
3 solver = SensitivitySolver()
4 res = solver.solve(model=mlsysim.Models.Language.
5                     Llama3_8B,
6                     hardware=mlsysim.Hardware.Cloud.
7                     A100)
8 print(res.sensitivities)
9 # {'peak_flops': 0.0, 'memory_bandwidth': -0.0875,
10 #  'memory_capacity': 0.0}
11 print(f"Binding: {res.binding_constraint}")
12 # Output: Binding: memory_bandwidth
```

Listing 4: **Sensitivity Analysis.** Identifying the binding constraint via 10% hardware perturbations.

The result is unambiguous: a 10% increase in HBM bandwidth cuts decode latency by 8.8%, whereas a 10% increase in peak FLOPS yields nothing at all, because batch-1 decode sits strictly on the memory roof of Equation (3). The `SynthesisSolver` then performs the inverse solve. Given a 50 ms inter-token latency SLA for LLaMA-70B, it synthesizes the minimum required bandwidth as $\text{BW}_{\text{req}} = |W|/T_{\text{target}} = 2800 \text{ GB/s}$, $1.4\times$ the A100’s 2 TB/s, confirming that LLaMA-70B inference is firmly in the memory-bound regime and that hardware procurement should prioritize HBM bandwidth over peak throughput.

7.3.3 R3: End-to-End LLaMA-70B Training Audit

To illustrate how `MLSYS·IM`’s solvers compose across all six taxonomy domains, we trace a complete training analysis for LLaMA-70B on 512 H100 GPUs (64 nodes $\times$ 8 GPUs, NVLink intra-node, 400 Gb/s InfiniBand inter-node) in Québec (20 gCO₂/kWh, PUE 1.06).

Node (Walls 1–3). With DP degree 64, each DP rank processes $4\text{M}/64 = 62,500$ tokens per step. Within each rank, TP=8 partitions the model across 8 local GPUs. The per-rank compute demand is $C_{\text{rank}} = 6 \times 70\text{B} \times 62,500 = 2.63 \times 10^{16}$ FLOPs. The 8-GPU TP group delivers $989 \times 10^{12} \times 0.4 \times 8 = 3.17 \times 10^{15}$ FLOP/s (at $\eta = 0.4$), yielding a per-step compute time of $T_{\text{compute}} = 2.63 \times 10^{16} / 3.17 \times 10^{15} = 8.3\text{s}$. The `SingleNodeModel` classifies this as compute-bound. Memory bandwidth ($8 \times 3.35 = 26.8 \text{ TB/s}$) can stream the 140 GB model weights in 5.2 ms, far below the 8.3 s compute time.

Data (Walls 8–10). The `DataModel` checks ingestion. At global batch size 4M tokens, the data pipeline must sustain $4\text{M} \times 2 \text{ bytes} / 8.3 \text{ s} \approx 0.96 \text{ MB/s}$ from storage, a trivially small demand. With NVMe delivering 6.5 GB/s per node across 64 nodes, the pipeline is not the bottleneck. (For LLM training, tokenized data is compact; the data wall binds primarily in vision tasks with large image payloads.)

Algorithm (Walls 11–13). The `ScalingModel` verifies that the training budget is compute-optimal. For $C = 2 \times 10^{24}$ FLOPs, the Chinchilla-optimal model size is $P^* \approx 130\text{B}$, indicating that 70B at this budget is slightly over-trained (more data per parameter than optimal), a deliberate choice for inference efficiency.

Fleet (Walls 14–16). The `DistributedModel` models communication. With TP=8 (intra-node NVLink, 900 GB/s) and DP=64 (inter-node IB, 50 GB/s per port), the DP AllReduce synchronizes $140 \text{ GB} / 8 = 17.5 \text{ GB}$ of gradients per TP rank across 64 nodes. Ring AllReduce cost is $2 \times (63/64) \times 17.5 \text{ GB} / 50 \text{ GB/s} \approx 689 \text{ ms}$. With $\eta_{\text{overlap}} = 0.9$, only $0.15 \times 689 = 103 \text{ ms}$ of communication is exposed, yielding a scaling efficiency of $8.3 / (8.3 +$

0.103) = 98.8% at 64 nodes. The `ReliabilityModel` estimates cluster MTBF = 10 000 hrs/512 $\approx$ 19.5 hrs, requiring hourly checkpoints.

Operations (Walls 17–20). The `EconomicsModel` projects a 30-day training run: CapEx (512 H100s at \$30K each) of \$15.4M amortized over 3 years yields a per-run allocation of $\$15.4\text{M} \times 30/1,095 \approx \422K , plus OpEx (power at $700\text{W} \times 512 \text{ GPUs} \times 720 \text{ hrs}$ at \$0.06/kWh) of \$15.5K, for a total run TCO of $\sim \$0.44\text{M}$. The `SustainabilityModel` estimates 274 MWh facility energy and 5.5 tonnes CO₂, 36 $\times$ less carbon than the same run in Iowa (680 gCO₂/kWh, PUE 1.12).

Analysis (Walls 21–22). The `SensitivitySolver` confirms the binding constraint is compute (Wall 1), not memory or communication, with $\partial T/\partial F_{\text{peak}} = -0.91$. The `SynthesisSolver` synthesizes the minimum hardware to complete training in 14 days: 1,024 GPUs, doubling DP to 128.

This end-to-end trace exercises 12 of the 22 walls through a single model, demonstrating how solver composition produces a complete system assessment from individual physics-based constraint equations.

7.3.4 R4: Automated Parallelism Search

A researcher needs to schedule a 175B-parameter model on a new 2,048-GPU cluster. Manually searching the 3D-parallelism space (TP $\times$ PP $\times$ DP) is error-prone. A split that maximizes DP might exceed the 80 GB HBM capacity, while a split that maximizes TP might saturate the NVLink interconnect. Instead of trial and error, the researcher invokes a Tier 3 Optimizer. They configure the `ParallelismOptimizer` with the workload and cluster, which maximizes MFU subject to a built-in memory screen: the per-GPU weight-plus-gradient shard must fit in 90% of HBM capacity ($\leq 72 \text{ GB}$). The optimizer sweeps power-of-two factorizations of 2,048 with TP capped at the node size, evaluating the `DistributedModel` at each of the 29 memory-feasible candidates. In well under a second, it returns the optimal schedule: TP=8, PP=2, DP=128, correctly deducing that TP must match the intra-node GPU count to avoid traversing the slower inter-node fabric, and that PP=2 is the minimum pipeline depth at which the $175\text{B} \times 2 \text{ bytes} \times 2/(8 \times 2) \approx 43.8 \text{ GB}$ shard fits the screen (PP=1 would demand 87.5 GB). This demonstrates the power of the “engineering engine” to invert the analytical models into automated design-space synthesis.

8 Fallacies & Pitfalls

Following the tradition of [Hennessy et al. \(2024\)](#), we highlight common misconceptions that MLSYS·IM is designed to expose.

Fallacy: Doubling peak FLOP/s halves training time. A student might assume that upgrading from A100 (312 TFLOP/s FP16 dense) to H100 (989 TFLOP/s, a 3.2 $\times$ increase) should yield a 3.2 $\times$ speedup. MLSYS·IM’s `SingleNodeModel` reveals why this is false. LLM inference is memory-bandwidth-bound, and the A100-to-H100 bandwidth improvement is only about 1.6 $\times$ (2 TB/s $\rightarrow$ 3.35 TB/s). For memory-bound workloads, training time scales with bandwidth, not FLOPS. The Roofline model (Equation (3)) makes this visible by showing that the binding constraint determines which hardware parameter matters.

Fallacy: Communication overhead is negligible at 512 GPUs. At 512 H100 GPUs (64 nodes), the DP AllReduce for a 70B model’s 17.5GB gradient shard costs 689 ms, of which only 103 ms is exposed after 85% compute–communication overlap, just 1.2% of the 8.3 s compute step. But the `ReliabilityModel` reveals the hidden cost. Cluster MTBF drops to ~ 20 hours, requiring hourly checkpoints that each pause training for 30–60 seconds. Over a 30-day run, checkpoint overhead (Wall 19) exceeds communication overhead (Wall 14) by 10 $\times$, a cost invisible to communication-only analysis.

Pitfall: Using peak bandwidth in back-of-the-envelope calculations. Vendor datasheets report peak HBM bandwidth (e.g., 3.35 TB/s for H100). In practice, sustained bandwidth under real workloads is 70–85% of peak due to bank conflicts, address patterns, and memory controller scheduling ([NVIDIA Corporation, 2023](#)). A student using peak bandwidth will underestimate LLM decode latency by 15–30%. MLSYS·IM’s MFU parameter (η) explicitly accounts for this gap, and the default ranges (Equation (6)) guide students toward realistic estimates.

Pitfall: Ignoring geography in carbon accounting. Two identical training runs produce vastly different environmental impact depending on grid carbon intensity. As demonstrated in Case I2, the same 256-GPU cluster emits 36 $\times$ more CO₂ in Iowa than in Québec. Students who omit Wall 18 (Sustainability) from their analysis miss a first-order systems design variable, one that increasingly affects both cost (carbon pricing) and regulatory compliance.

Fallacy: Quantization always provides a linear speedup. Reducing precision from FP16 to INT4 ($r_{\text{quant}} = 4\times$) reduces memory by 4 $\times$, but the compute speedup depends on whether the workload is memory-bound or compute-bound. For memory-bound LLM decode, the 4 $\times$ memory reduction translates to nearly 4 $\times$ throughput improvement because the bottleneck is weight reads. For compute-bound training, the same quantization provides zero throughput benefit because compute, not memory, is the ceiling. MLSYS·IM’s

Roofline analysis makes this regime-dependent behavior explicit.

9 Discussion & Limitations

“All models are wrong, but some are useful” (Box, 1976). Users must understand the boundaries of MLSYS·IM’s analytical abstraction. We organize the limitations into modeling scope, accuracy trade-offs, and pedagogical implications, then outline future directions.

9.1 What MLSYS·IM Cannot Model

No microarchitectural effects. MLSYS·IM has no notion of L1/L2 cache hierarchies, branch prediction, warp scheduling, or register pressure. These second-order effects are absorbed into a single scalar efficiency parameter (η , the ratio of sustained to peak FLOP/s). While η provides a serviceable approximation for back-of-the-envelope reasoning, it cannot capture workload-dependent microarchitectural behavior; a matrix multiply and a sparse attention kernel may achieve very different η on identical silicon.

No real network congestion. The communication model uses the classical α - B_{link} formulation (latency plus inverse-bandwidth), which assumes dedicated links. MLSYS·IM does not model adaptive routing, network contention under multi-tenant traffic, or congestion collapse, phenomena that become critical at scales beyond $\sim 10,000$ nodes, precisely the regime where ASTRA-sim 2.0 (Won et al., 2023) provides essential fidelity. The static bisection view behind Wall 10 (Topology) is optimistic under the same cross-traffic conditions.

No OS/runtime overhead. Kernel launch latency, CUDA stream scheduling, Python GIL contention, and host-device transfer overhead are absent. For inference-dominated workloads where kernel launch time can rival compute time, this omission can meaningfully affect predictions.

No heterogeneous fleets. The DistributedModel assumes homogeneous nodes, where all accelerators in a fleet share the same compute, memory, and interconnect specifications. Production clusters increasingly mix hardware generations (e.g., A100 and H100 nodes in the same job), and fleet-level efficiency metrics such as ML Productivity Goodput (Wongpanich et al., 2025) capture this heterogeneity. Modeling heterogeneous fleets would require per-node load balancing and straggler analysis beyond the current analytical framework.

No dynamic behavior. MLSYS·IM models steady-state throughput. Transient effects (thermal throttling, dynamic clock boosting, memory fragmentation over long training runs, and checkpoint I/O bursts) are outside its scope. A training run that degrades over 72

hours due to thermal saturation will appear identical to one that sustains peak throughput.

Heuristic accuracy models. The accuracy degradation curves in CompressionModel are heuristic step functions (for quantization) and exponentials (for pruning), not architecture-specific empirical fits. Real accuracy loss depends on model architecture, calibration methodology, and quantization method (e.g., well-calibrated GPTQ INT4 can achieve $<0.5\%$ degradation, while naive round-to-nearest may lose $10\%+$). Users should treat these curves as directional indicators, not ground truth.

9.2 Walls Not Included

The 22-wall taxonomy is comprehensive but not exhaustive. Several constraints were considered and excluded, each for a specific reason.

Thermal throttling. Sustained power density can force throughput below peak TDP, but this is absorbed into η rather than modeled as a distinct wall.

Resource fragmentation. Scattered GPU availability across nodes prevents job scheduling even when aggregate capacity is sufficient; this is a combinatorial bin-packing problem beyond the current analytical framework.

Compiler/graph optimization. The gap between a framework’s computational graph and the executed kernel schedule affects both latency and MFU, but varies too rapidly across software versions to model analytically.

The selection criterion for inclusion was: does the constraint have a stable, published analytical formulation that remains valid across hardware generations? Constraints requiring empirical trace data or combinatorial optimization were deferred to future work.

9.3 The Accuracy–Speed Trade-off

Each omission above reflects the same trade-off: three orders of magnitude improvement in evaluation speed at the cost of second-order fidelity. The precedent is the MIPS/SPIM simulator (Hennessy et al., 2024), which models pipeline hazards and stalls but omits superscalar execution and cache hierarchies, prioritizing pedagogical clarity over the full complexity of commercial processors. MLSYS·IM applies the same philosophy to ML systems, making quantitative reasoning accessible to students who may never operate a production cluster.

The relevant question is not “How accurate is MLSYS·IM?” but “Does it identify the correct binding constraint?” A first-order model that correctly determines whether a system is memory-bound, compute-bound, or network-bound provides actionable architectural insight even when its absolute latency prediction is $\pm 20\%$ from a cycle-accurate trace. The binding constraint dictates which hardware investment

yields the largest return, and this ordinal ranking is far more robust than cardinal predictions. Practitioners who know that their system is memory-bandwidth-bound will invest in higher-bandwidth memory regardless of whether the predicted latency is 47 ms or 53 ms.

9.4 Future Work

We identify several directions for extending MLSYS·IM.

Broader empirical validation. Section 6 validates against seven anchors spanning five domains. Future work will extend this to newer hardware generations already present in the registry (NVIDIA B200/GB200, TPU v6) as published benchmark results appear, inference serving under load (continuous batching with realistic request distributions), and checkpoint overhead at scale, where published data points are becoming available from reproducibility studies.

Community hardware registry. The **Hardware** registry currently contains a curated set of hardware entries verified against manufacturer datasheets. We plan to open contributions from the community, with automated verification scripts that cross-check submitted specifications against known physical limits (e.g., memory bandwidth cannot exceed pin count $\times$ data rate).

Custom degradation curves. Future versions will allow users to supply empirically fitted accuracy-compression curves from their own quantization experiments, replacing the current heuristic polynomials with data-grounded models.

TinyTorch integration. MLSYS·IM provides analytical predictions; TinyTorch (Reddi et al., 2025b), the companion educational framework, provides implementation-based verification. Connecting the two tools creates a predict-then-verify loop. Students estimate training time and memory consumption in MLSYS·IM, then run the actual training in TinyTorch and compare. This closed loop reinforces quantitative reasoning by grounding analytical models in empirical observation.

Expanding Tier 3 Optimizers (Pareto Frontiers). Currently, the Tier 3 optimizers search single-dimensional objective spaces (e.g., maximizing MFU or maximizing batch size under a latency constraint). Future work will extend the Tier 3 engine to support multi-objective Pareto frontiers, simultaneously optimizing across latency, total cost, carbon footprint, and accuracy. This will enable richer design-space exploration and formally expose the inherent tensions between performance and sustainability.

9.5 The Pedagogical Argument

Even when MLSYS·IM’s predictions deviate by 20% from measured values, the pedagogical value lies in the

reasoning process rather than the numerical output. A student who sweeps 1,000 configurations and identifies memory bandwidth as the binding constraint for LLM inference has acquired a transferable analytical skill, that of determining which resource limits performance. The framework trains students to formulate the correct quantitative questions (arithmetic intensity, ridge point location, communication-to-computation ratio) and these questions generalize to production systems even as specific hardware parameters change across generations.

10 Conclusion

Modern ML systems are no longer limited by a single resource that can be optimized in isolation. A deployment decision can hinge simultaneously on arithmetic intensity, memory capacity, communication topology, queueing delay, energy price, regional carbon intensity, and failure recovery. This coupling makes intuition brittle: a faster accelerator may not improve decode latency, a larger batch may move the bottleneck from memory to compute, and a cheaper datacenter may be the wrong choice once emissions or water constraints are included. The practical challenge is to make these interactions visible before teams commit to hardware, cloud spend, or weeks of empirical benchmarking.

MLSYS·IM treats ML system design as a set of executable constraints. Its purpose is not to replace profilers, trace-driven simulators, or cycle-level models; those tools remain necessary when the question is precise performance on a concrete system. Instead, MLSYS·IM occupies the earlier stage where engineers and students need to ask which wall binds, which parameter matters, and which design alternatives are worth measuring at all. Its standard is first-order fidelity: close enough to identify bottlenecks and rank alternatives, explicit enough to expose assumptions, and fast enough to explore the design space before measurement begins. By separating demand from supply, carrying units through every calculation, and composing first-principles resolvers across the full stack, it turns informal back-of-the-envelope reasoning into a reproducible infrastructure model.

That artifact is also pedagogical. Paired with TinyTorch (Reddi et al., 2025b) and the open *Machine Learning Systems* text (Reddi et al., 2025a), MLSYS·IM gives learners a way to connect equations, code, and infrastructure consequences without requiring access to a GPU cluster. The goal is not merely to compute a latency or carbon number, but to teach a durable habit of systems reasoning: identify the binding constraint, check the units, perturb the design, and explain why the result changed. As ML systems continue to scale across larger models, heterogeneous fleets, and stricter economic and environ-

mental constraints, that habit will matter as much as any single modeling tool.

Generative AI Usage Statement

The author used a variety of generative AI tools during the development of MLSYS-IM and the preparation of this manuscript. These tools supported literature discovery, landscape exploration, brainstorming, code prototyping, implementation iteration, prose editing, figure refinement, and pedagogical organization. The author designed the framework, taxonomy, registry hierarchy, modeling abstractions, evaluation methodology, and educational framing; reviewed and revised AI-assisted outputs; and verified the final code, equations, citations, figures, and claims. Generative AI tools were not treated as authors or sources of scientific authority. The author remains fully responsible for the originality, correctness, and integrity of the work.

References

- Martin Abadi, Andy Chu, Ian Goodfellow, et al. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 308–318. ACM, 2016. doi: 10.1145/2976749.2978318. URL <https://doi.org/10.1145/2976749.2978318>.
- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134. USENIX Association, July 2024a. ISBN 978-1-939133-40-3. URL <https://www.usenix.org/conference/osdi24/presentation/agrawal>.
- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Vidur: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems (MLSys)*. mlsys.org, 2024b. URL <https://arxiv.org/abs/2405.05465>.
- Abhimanyu Bambhaniya, Ritik Raj, Geonhwa Jeong, Souvik Kundu, Sudarshan Srinivasan, Midhilesh Elavazhagan, Madhu Kumar, and Tushar Krishna. Demystifying platform requirements for diverse LLM inference use cases. *arXiv preprint arXiv:2406.01698*, 2024.
- Luiz André Barroso and Urs Hölzle. The case for energy-proportional computing. *Computer*, 40(12):33–37, 2007. ISSN 0018-9162. doi: 10.1109/mc.2007.443. URL <https://doi.org/10.1109/mc.2007.443>.
- Luiz André Barroso, Urs Hölzle, and Parthasarathy Ranganathan. *The Datacenter as a Computer*. Synthesis Lectures on Computer Architecture. Springer International Publishing, 3rd edition, 2018. ISBN 9783031006333, 9783031017612. doi: 10.1007/978-3-031-01761-2. URL <https://doi.org/10.1007/978-3-031-01761-2>.
- Nathan Binkert, Bradford Beckmann, Gabriel Black, et al. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7, 2011. ISSN 0163-5964. doi: 10.1145/2024716.2024718. URL <https://doi.org/10.1145/2024716.2024718>.
- George E. P. Box. Science and statistics. *J. Am. Stat. Assoc.*, 71(356):791–799, None 1976. ISSN 0162-1459, 1537-274X. doi: 10.1080/01621459.1976.10480949. URL <https://doi.org/10.1080/01621459.1976.10480949>.
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, et al. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- Russ Cox, M. Frans Kaashoek, and Robert Morris. xv6: A simple, Unix-like teaching operating system. MIT PDOS, 2011. URL <https://pdos.csail.mit.edu/6.828/xv6>.
- John T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Gener. Comput. Syst.*, 22(3):303–312, 2006. ISSN 0167-739X. doi: 10.1016/j.future.2004.11.016. URL <https://doi.org/10.1016/j.future.2004.11.016>.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems 35*, volume 35, pages 16344–16359. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022. doi: 10.522/02/068431-1189. URL <https://doi.org/10.522/02/068431-1189>.
- Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013. ISSN 0001-0782, 1557-7317. doi: 10.1145/2408776.2408794. URL <https://doi.org/10.1145/2408776.2408794>.
- Jeffrey Dean, Greg S. Corrado, Rajat Monga, et al. Large scale distributed deep networks. *Advances in Neural Information Processing Systems*, 25:1223–1231, 2012.

- DeepSeek-AI. Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pages 1731–1745. ACM, 2025. doi: 10.1145/3695053.3731412. URL <https://doi.org/10.1145/3695053.3731412>.
- Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. Check-n-run: a checkpointing system for training deep learning recommendation models. In *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2022.
- Ahmad Faiz, Sotaro Kaneda, Ruhan Wang, Rita Osi, Prateek Sharma, Fan Chen, and Lei Jiang. LLMCarbon: Modeling the end-to-end carbon footprint of large language models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2024.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL <https://www.jmlr.org/papers/v23/21-0998.html>.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.
- Amir Gholami, Sehoon Kim, Zhen Dong, et al. *A Survey of Quantization Methods for Efficient Neural Network Inference*, pages 291–326. Chapman and Hall/CRC, 2021. ISBN 9781003162810. doi: 10.1201/9781003162810-13. URL <https://doi.org/10.1201/9781003162810-13>.
- Udit Gupta, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. Chasing carbon: The elusive environmental footprint of computing. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 85–99. IEEE, 2022. doi: 10.1109/hpca53966.2022.00013. URL <https://doi.org/10.1109/hpca53966.2022.00013>.
- Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *Proceedings of the 4th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2016.
- John L. Hennessy, David A. Patterson, and Christos Kozyrakis. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 7th edition, 2024. ISBN 978-0443154065.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems 35*, volume 35, pages 30016–30030. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022. doi: 10.52202/068431-2176. URL <https://doi.org/10.52202/068431-2176>.
- Mikhail Isaev, Nic McDonald, Larry Dennison, and Richard Vuduc. Calculon: A methodology and tool for high-level co-design of systems and large language models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–14. ACM, 2023. doi: 10.1145/3581784.3607102. URL <https://doi.org/10.1145/3581784.3607102>.
- Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond data and model parallelism for deep neural networks. In *Proceedings of Machine Learning and Systems (MLSys)*. mlsys.org, 2019. URL <https://arxiv.org/abs/1807.05358>.
- Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems*, 5, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/80083951326cf5b35e5100260d64ed81-Abstract-mlsys2023.html.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Charles E. Leiserson. Fat-trees: Universal networks for hardware-efficient supercomputing. *IEEE Transactions on Computers*, C-34(10):892–901, 1985. ISSN 0018-9340. doi: 10.1109/tc.1985.6312192. URL <https://doi.org/10.1109/tc.1985.6312192>.
- Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint*

- arXiv:2006.16668, 2020. URL <https://arxiv.org/abs/2006.16668>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 2023.
- Cheng Li. llm-analysis: Latency and memory analysis of transformer models. <https://github.com/cli99/llm-analysis>, 2023. Accessed: 2025-01-15.
- Mingyu Liang, Hiwot Tadesse Kassa, Wenyin Fu, Brian Coutinho, Louis Feng, and Christina Delimitrou. Lumos: Efficient performance modeling and estimation for large-scale LLM training. In *Proceedings of Machine Learning and Systems (MLSys)*. mlsys.org, 2025.
- Sean Lie. Cerebras architecture deep dive: First look inside the HW/SW co-design for deep learning. In *IEEE Hot Chips 34 Symposium*. IEEE, 2022.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *GetMobile: Mobile Computing and Communications*, 28(4):12–17, 2024. ISSN 2375-0529, 2375-0537. doi: 10.1145/3714983.3714987. URL <https://doi.org/10.1145/3714983.3714987>.
- John D. C. Little. A proof for the queuing formula: $\langle i \rangle \langle l \rangle = \Lambda \langle i \rangle \langle w \rangle$. *Oper. Res.*, 9(3):383–387, 1961. ISSN 0030-364X, 1526-5463. doi: 10.1287/opre.9.3.383. URL <https://doi.org/10.1287/opre.9.3.383>.
- Llama Team, AI Meta. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Kadan Lottick, Silvia Susai, Sorelle A. Friedler, and Jonathan P. Wilson. Energy usage reports: Environmental awareness as part of algorithmic accountability. *arXiv preprint arXiv:1911.08354*, 2019.
- Peter Mattson, Christine Cheng, Gregory Diamos, et al. Mlperf: An industry standard benchmark suite for machine learning performance. *IEEE Micro*, 40(2):8–16, 2020. ISSN 0272-1732, 1937-4143. doi: 10.1109/mm.2020.2974843. URL <https://doi.org/10.1109/mm.2020.2974843>.
- Jayashree Mohan, Amar Phanishayee, Ashish Raniwala, and Vijay Chidambaram. Analyzing and mitigating data stalls in dnn training. *Proc. VLDB Endow.*, 14(5): 771–784, 2021. ISSN 2150-8097. doi: 10.14778/3446095.3446100. URL <https://doi.org/10.14778/3446095.3446100>.
- Derek G. Murray, Jiri Simsa, Ana Klimovic, and Ihor Indyk. tf.data: A machine learning data processing framework. *Proceedings of the VLDB Endowment*, 14(12):2945–2958, 2021.
- Deepak Narayanan, Mohammad Shoeybi, Jared Casper, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. ACM, 2021. doi: 10.1145/3458817.3476209. URL <https://doi.org/10.1145/3458817.3476209>.
- NVIDIA Corporation. NVIDIA H100 Tensor Core GPU datasheet. <https://www.nvidia.com/en-us/data-center/h100/>, 2023. Accessed: 2024-06-15.
- Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Joel Emer, et al. Timeloop: A systematic approach to dnn accelerator evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 304–315. IEEE, 2019. doi: 10.1109/ispass.2019.00042. URL <https://doi.org/10.1109/ispass.2019.00042>.
- Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024. doi: 10.1109/isca59077.2024.00019. URL <https://doi.org/10.1109/isca59077.2024.00019>.
- David Patterson, Joseph Gonzalez, Quoc Le, et al. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021.
- David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 5th edition, 2014. ISBN 978-0124077263.
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, et al. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5. mlsys.org, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html.
- Hang Qi, Evan R. Sparks, and Ameet Talwalkar. PALEO: A performance model for deep neural networks. In

- Proceedings of the 5th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2017.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020. doi: 10.1109/sc41405.2020.00024. URL <https://doi.org/10.1109/sc41405.2020.00024>.
- Vijay Janapa Reddi et al. *Machine Learning Systems: Principles and Practices of Engineering Artificially Intelligent Systems*. Harvard University, 2025a. URL <https://mlsysbook.ai>.
- Vijay Janapa Reddi et al. TinyTorch: A progressive educational framework for machine learning systems. Harvard University, 2025b. URL <https://mlsysbook.ai/tinytorch>.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2017.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2025.
- Arthur G. Stephenson, Lia S. LaPiana, Daniel R. Mulville, Peter J. Rutledge, Frank H. Bauer, David Folta, Greg A. Dukeman, Robert Sackheim, and Peter Norvig. Mars Climate Orbiter Mishap Investigation Board Phase I Report. Technical report, National Aeronautics and Space Administration, None 1999.
- Rich Sutton. The bitter lesson. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>, 2019. Accessed: 2024-06-15.
- Andrew S. Tanenbaum and Albert S. Woodhull. *Operating Systems: Design and Implementation*. Prentice Hall, 3rd edition, 2006.
- Xizheng Wang et al. SimAI: Unifying architecture design and performance tuning for large-scale large language model training with scalability and precision. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2025.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline. *Communications of the ACM*, 52(4):65–76, 2009. ISSN 0001-0782, 1557-7317. doi: 10.1145/1498765.1498785. URL <https://doi.org/10.1145/1498765.1498785>.
- William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. Astra-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 283–294. IEEE, 2023. doi: 10.1109/ispass57527.2023.00035. URL <https://doi.org/10.1109/ispass57527.2023.00035>.
- Arissa Wongpanich, Tayo Oguntebi, Jose Baiocchi Paredes, Yu Emma Wang, Phitchaya Mangpo Phothilimthana, Ritwika Mitra, Zongwei Zhou, Naveen Kumar, and Vijay Janapa Reddi. Machine learning fleet efficiency: Analyzing and optimizing large-scale Google TPU systems with ML productivity goodput, 2025. arXiv:2502.06982.
- Yannan Nellie Wu, Joel S. Emer, and Vivienne Sze. Accelerator: An architecture-level energy estimation methodology for accelerator designs. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8. IEEE, 2019. doi: 10.1109/iccad45719.2019.8942149. URL <https://doi.org/10.1109/iccad45719.2019.8942149>.
- John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974. ISSN 0001-0782, 1557-7317. doi: 10.1145/361147.361115. URL <https://doi.org/10.1145/361147.361115>.
- Geoffrey X. Yu, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. Habitat: A runtime-based computational performance predictor for deep neural network training. In *Proceedings of the 2021 USENIX Annual Technical Conference (ATC)*. USENIX Association, 2021.
- Zhihang Yuan, Yuzhang Shang, Yang Zhou, et al. LLM inference unveiled: Survey and roofline model insights. *arXiv preprint arXiv:2402.16363*, 2024.
- Hengrui Zhang, August Ning, Rohan Baskar Prabhakar, and David Wentzlaff. Llmcompass: Enabling efficient hardware design for large language model inference.

In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 1080–1096. IEEE, 2024. doi: 10.1109/isca59077.2024.00082. URL <https://doi.org/10.1109/isca59077.2024.00082>.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37*, pages 62557–62583. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi: 10.52202/079017-2000. URL <https://doi.org/10.52202/079017-2000>.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210. USENIX Association, July 2024. ISBN 978-1-939133-40-3. URL <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.

A Registry Snapshot

The MLSys Zoo comprises eight registries spanning the full systems stack (plus the `Literature` and `ReferenceStats` auxiliary surfaces). This appendix is a representative snapshot of the current local registry surface: enough to show the kinds of systems `MLSYS-IM` can compose, without treating the paper as the authoritative inventory. The typed Python and YAML registry source remains the source of truth for the complete, up-to-date entries.

A.1 Hardware Registry

Table 7 summarizes the current accelerator coverage by deployment tier. Each entry is a fully typed `HardwareNode` with pint-dimensional fields. Users add custom entries by instantiating the same types (Listing 2), and custom entries participate in the same dimensional pipeline as vetted ones. For example, a research team evaluating a pre-release accelerator can define a speculative entry with projected specifications and immediately sweep it against existing workloads (Listing 5).

```
1 from mlsysim.hardware.types import HardwareNode, ComputeCore, MemoryHierarchy
2 from mlsysim.core.units import TFLOPs, GB, TB, second, watt
3
4 custom_accelerator = HardwareNode(
5     name="Speculative GPU 2027",
6     release_year=2027,
7     compute=ComputeCore(
8         peak_flops=1200 * TFLOPs / second,
9         precision_flops={"fp16": 1200 * TFLOPs / second, "fp8": 2400 * TFLOPs / second}
10    ),
11     memory=MemoryHierarchy(
12         capacity=144 * GB,
13         bandwidth=5.0 * TB / second
14    ),
15     tdp=800 * watt
16 )
```

Listing 5: **Extending the Registry.** Defining a custom, speculative hardware entry. Because it uses the typed `HardwareNode` schema (Table 10), it can immediately be evaluated by any Tier 1 or Tier 2 solver without risking unit mismatches.

³*CS-3 BW is on-wafer SRAM bandwidth (21 PB/s). The weight-streaming serving case study (Section 7) uses the ~1.2TB/s MemoryX injection bandwidth, which is the effective off-chip bandwidth seen by the host. [†]GB200 NVL72 is a rack-scale system entry (72 Blackwell GPUs behind one NVLink switch domain) rather than a single accelerator.

Table 7: **Hardware Registry Snapshot.** The 37 hardware entries currently present in the local registry, organized by deployment tier. Peak FLOP/s is FP16 Tensor Core (or equivalent) throughput; BW is primary-memory bandwidth (HBM, LPDDR, or on-die SRAM as appropriate). Memory uses the marketed (binary) capacity labels.³ The last row shows a user-defined custom entry.

Tier	Accelerator	Peak (TFLOP/s)	Mem (GB)	BW (TB/s)	TDP (W)	Interconnect
<i>Cloud</i>						
	NVIDIA T4	65	16	0.32	70	PCIe Gen3
	NVIDIA V100	125	32	0.90	300	NVLink 2.0 (300 GB/s)
	NVIDIA V100 (PCIe)	112	32	0.90	250	PCIe Gen3
	NVIDIA A100	312	80	2.04	400	NVLink 3.0 (600 GB/s)
	NVIDIA H100	989	80	3.35	700	NVLink 4.0 (900 GB/s)
	NVIDIA H200	989	141	4.80	700	NVLink 4.0 (900 GB/s)
	NVIDIA B200	2,250	192	8.00	1,000	NVLink 5.0 (1,800 GB/s)
	NVIDIA GB200 NVL72 [†]	162,000	13,800	576	120,000	NVLink Switch
	AMD MI250X	383	128	3.20	500	Infinity Fabric
	AMD MI300X	1,307	192	5.30	750	Infinity Fabric
	Intel Gaudi 2	432	96	2.45	600	RoCE (24×100 GbE)
	Intel Gaudi 3	1,835	128	3.70	900	RoCE
	AWS Trainium 2	380	96	2.40	500	NeuronLink
	Google TPU v1	92 (INT8)	8	0.034	75	PCIe
	Google TPU v2	45	16	0.70	200	ICI
	Google TPU v3	105	32	0.90	250	ICI
	Google TPU v4	275	32	1.20	200	ICI (300 GB/s)
	Google TPU v5p	459	95	2.76	300	ICI (1,200 GB/s)
	Google TPU v6	918	32	1.60	300	ICI
	Cerebras CS-3*	125,000	44	21,000	23,000	MemoryX (1,200 GB/s)
	Intel SGX Enclave	1	0.128	0.01	—	—
	Reference CPU Server	1	64	0.05	150	—
<i>Workstation</i>						
	NVIDIA DGX Spark	125	128	0.27	200	—
	Apple M3 Max (MacBook)	14	128	0.40	100	Unified memory
<i>Edge</i>						
	NVIDIA Jetson AGX Orin	275	64	0.20	60	PCIe Gen4
	NVIDIA Jetson Orin NX	25	16	0.10	25	PCIe Gen4
	NVIDIA Jetson Orin Nano	10	8	0.07	15	PCIe Gen4
	Google Coral Edge TPU	4	1	0.008	2	USB 3.0
	Intel NUC + Movidius	1	16	0.025	15	USB 3.0
	Generic x86 Server	1	128	0.10	300	—
<i>Mobile</i>						
	Apple A17 Pro (iPhone 15 Pro)	35	8	0.10	5	Unified memory
	Google Tensor G3 (Pixel 8)	15	8	0.06	5	LPDDR5X
	Snapdragon 8 Gen 3	45	12	0.08	5	LPDDR5X
	Apple M2	16	16	0.10	20	Unified memory
<i>Tiny</i>						
	ESP32-S3	0.0005	0.004	0.0001	0.4	SPI
	Himax WE-1	0.0002	0.002	0.0001	0.005	SPI
	Nordic nRF52840	0.0001	0.001	0.0001	0.015	SPI
<i>Custom (user-defined)</i>						
	Custom Accelerator	500	96	4.00	500	—

Table 8: **Infrastructure and Systems Registry Snapshot.** *Left:* Current grid profiles for sustainability analysis (CI = carbon intensity in gCO₂/kWh; PUE = power usage effectiveness; WUE = water usage effectiveness in L/kWh). *Right:* Representative compute nodes, summarized network fabrics, and current fleet tiers for distributed analysis.

Grid Profiles (12 registered)					Nodes, Fabrics, and Fleets	
Profile	CI	PUE	WUE	Source	Compute Nodes	
Norway	10	1.06	0.0	Hydro	DGX A100	8× A100, NVLink 3.0, 600 GB/s
Québec	20	1.06	0.0	Hydro	DGX H100	8× H100, NVLink 4.0, 900 GB/s
Iceland	28	1.06	0.0	Geo/Hydro	DGX B200	8× B200, NVLink 5.0, 1,800 GB/s
France	50	1.12	1.8	Nuclear	Kempner H100	4× H100 academic server
EU Average	270	1.12	1.8	Mixed		
Germany	385	1.12	1.8	Mixed	Network Fabrics (10 registered)	
Texas	400	1.12	1.8	Mixed	IB HDR / NDR / XDR / GXDR	200 / 400 / 800 / 1,600 Gb/s, 5–7 μs
US Average	429	1.12	1.8	Mixed		
China Average	555	1.40	1.8	Coal-heavy	Ethernet 10G–1.6T	10–1,600 Gb/s, 50 μs
Iowa (Coal/Gas)	680	1.12	1.8	Fossil	RoCE 100G	100 Gb/s, 10 μs
India Average	720	1.58	1.8	Coal-heavy	Fleet Configurations (11 registered)	
Poland (Coal)	820	1.58	1.8	Coal	Lab (64 GPU)	
					Research (256 GPU)	
					Kempner (384 GPU)	
					Training (512 GPU / 1K / 1K A100)	
					Production (2K GPU)	
					Frontier (8K GPU)	
					Training (10K) / Reference (25K)	
					Mega (100K GPU)	

A.2 Infrastructure and Systems Registries

The Hardware registry (Table 7) captures individual accelerator capabilities, but real deployments compose accelerators into nodes, connect nodes through network fabrics, and situate the resulting clusters in datacenters whose geography determines energy cost, carbon intensity, and cooling strategy. The Infrastructure and Systems registries complete this picture: Infrastructure encodes *where* the fleet runs, and Systems encodes *how* accelerators are assembled into distributed configurations. Together the three registries let a single resolver chain evaluate end-to-end metrics from per-device roofline through fleet-level TCO and carbon footprint.

The **Infrastructure** registry encodes regional environmental context for sustainability analysis, plus datacenter profiles, rack power envelopes, facility cooling, cloud pricing, and capacity lead times. Carbon intensity varies by more than 80× across the 12 registered grid profiles (10 gCO₂/kWh in Norway vs. 820 in Poland), making geography a first-order design variable (Case I2, Section 7). The **Systems** registry defines compute nodes, network fabrics, fleet configurations, TPU pods, switch fabrics, and 14 storage-related profiles and checkpoint-path anchors (NVMe generations, SATA, HDD, object stores, parallel file systems) used by the `DistributedModel`, `ReliabilityModel`, and `DataModel`. Table 8 presents representative entries from both registries.

A.3 Type System and Physics API

To support the separation of demand and supply, `MLSYS·IM` relies on a shallow but rigorously typed hierarchy of Pydantic models (Table 10). These types act as the contract between the declarative registries and the analytical solvers. Because all physical quantities (bandwidth, latency, power) are dimensioned using the `pint` library, users can extend any registry by instantiating these types directly without risking unit mismatch errors.

Beneath the resolvers lies the Physics Engine, which contains the dimensioned mathematical identities that govern ML systems. Table 9 catalogs the public API of these physics modules. Each function is stateless, accepting dimensioned quantities (or raw floats that are immediately coerced to units) and returning dimensioned

Table 9: Physics Module Public API. Complete function signatures organized by domain module. Every function accepts `pint Quantity` arguments (or raw floats coerced via `_ensure_unit()`) and returns dimensioned quantities. Import from `mlsysim.physics` (one module per domain).

Module	Function	Signature (key parameters → return)
performance	<code>dTime</code>	<code>(total_ops, n_devices, peak_flops, η) → seconds</code>
	<code>calc_bottleneck</code>	<code>(ops, model_bytes, device_flops, device_bw) → {time, regime}</code>
	<code>calc_amdahls_speedup</code>	<code>(parallel_frac, speedup_factor) → overall speedup</code>
	<code>calc_pipeline_bubble</code>	<code>(n_stages, n_microbatches, v_stages) → bubble fraction</code>
	<code>calc_effective_flops</code>	<code>(peak_flops, η) → sustained FLOP/s</code>
	<code>calc_training_time_days</code>	<code>(total_ops, n_devices, peak_flops, η) → days</code>
memory	<code>model_memory</code>	<code>(params, bytes_per_param, unit) → memory</code>
	<code>calc_activation_memory</code>	<code>(batch, seq, hidden, layers, prec) → bytes</code>
	<code>calc_kv_cache_size</code>	<code>(layers, heads, dim, seq, batch, prec) → bytes</code>
	<code>calc_paged_kv_cache_size</code>	<code>(same + page_size) → bytes</code>
	<code>calc_checkpoint_size</code>	<code>(n_params, bytes_per_param) → bytes</code>
serving	<code>calc_queue_latency_mmc</code>	<code>(arrival_rate, service_rate, n_servers) → wait time</code>
communication	<code>calc_ring_allreduce_time</code>	<code>(msg_bytes, n_gpus, bw, latency) → seconds</code>
	<code>calc_tree_allreduce_time</code>	<code>(msg_bytes, n_gpus, bw, latency) → seconds</code>
	<code>calc_all_to_all_time</code>	<code>(msg_bytes, n_gpus, bw, latency) → seconds</code>
	<code>calc_hierarchical_allreduce_time</code>	<code>(msg, intra_gpus, intra_bw, inter_nodes, inter_bw, α) → seconds</code>
reliability	<code>calc_young_daly_interval</code>	<code>(ckpt_cost, mtbf) → optimal interval</code>
	<code>calc_mtbf_cluster</code>	<code>(component_mtbf, n, correlation) → cluster MTBF</code>
	<code>calc_mtbf_node</code>	<code>(component_mtbf) → node MTBF</code>
	<code>calc_availability_stacked</code>	<code>(single_avail, n_replicas) → stacked availability</code>
	<code>calc_failure_probability</code>	<code>(mtbf, duration) → probability</code>
transformer	<code>calc_transformer_training_flops</code>	<code>(n_params, n_tokens) → total FLOPs</code>
	<code>calc_transformer_decode_flops</code>	<code>(n_params, n_tokens) → decode FLOPs</code>
economics	<code>calc_fleet_tco</code>	<code>(unit_cost, power, quantity, years, kwh_price) → TCO</code>
	<code>calc_monthly_egress_cost</code>	<code>(bytes_per_sec, cost_per_gb) → monthly cost</code>
networking	<code>calc_network_latency_ms</code>	<code>(distance_km) → latency</code>
statistics	<code>calc_population_stability_index</code>	<code>(expected, actual) → PSI</code>
	<code>calc_two_proportion_sample_size</code>	<code>(baseline, lift, z_α, z_β) → n</code>
	<code>calc_constraint_propagation_factor</code>	<code>(from, to, base) → factor</code>
quantities	<code>transfer_time / compute_time</code>	<code>(payload, bandwidth) / (work, throughput) → time</code>
	<code>energy_from_power / carbon_from_energy</code>	<code>(power, duration) / (energy, CI) → energy / mass</code>
	<code>memory_from_params / token_throughput</code>	<code>(params, bytes/param) / (tokens, duration) → bytes / rate</code>

bounds. These functions implement the core logic for the Iron Law of training, the Roofline model, and network communication topologies.

Table 10: **Core Type Schema.** Pydantic BaseModel types that compose registry entries. The hierarchy is deliberately shallow: HardwareNode aggregates ComputeCore and MemoryHierarchy as direct fields, not through deep inheritance. All physical fields are pint Quantity objects. Users extend any registry by instantiating these types directly (Listing 2).

Type	Key Fields
ComputeCore	peak_flops (FLOP/s), precision_flops (dict: fp8/fp16/tf32/int8 → FLOP/s), sm_count
MemoryHierarchy	capacity (bytes), bandwidth (bytes/s); optional SRAM, flash, and L2 tiers
IOInterconnect	bandwidth (bytes/s), latency (seconds), name
HardwareNode	compute, memory, storage, interconnect, nvlink, tdp (W), unit_cost (USD), dispatch_tax (s), metadata.provenance
ComputationGraph	total_ops (FLOPs), weight_bytes (bytes), arithmetic_intensity (FLOP/byte), parameter_count, layers
TransformerWorkload	name, architecture, parameters, layers, hidden_dim, heads, kv_heads
GridProfile	carbon_intensity_g_kwh, pue, wue (L/kWh), renewable_pct
NetworkFabric	topology, bandwidth (bytes/s), latency (s), oversubscription_ratio
Fleet	node, count, fabric (NetworkFabric), region/datacenter, mtbf_hours
SystemEvaluation	feasibility, performance, macro (three-level scorecard)

A.4 Code Examples Cookbook

To demonstrate how the theoretical 3-tier architecture from Section 5 is exposed in Python, we provide eight complete, runnable examples. Each example is written as a small engineering task: the prose names the question, the listing shows which registry facts are selected, and the returned object exposes the bound, bottleneck, or scorecard used to make a decision. The point is not to hide the model behind an API call, but to show how registry-backed composition turns a systems question into a repeatable calculation.

A.4.1 Scenario A: Rapid Parametric Sweeps (Tier 1)

Scenario: You want to know how much decode throughput continuous batching can buy on an H100 before KV-cache and activation traffic saturate the memory system. The code selects a workload from Models.Language and a supply point from Hardware.Cloud, then calls Engine.solve() once per batch size. Each returned performance object reports throughput, MFU, and the binding Roofline regime, so the sweep shows not only that throughput saturates but why it remains memory-bound.

```

1 from mlsysim.engine.engine import Engine
2 from mlsysim.hardware.registry import Hardware
3 from mlsysim.models.registry import Models
4 import pandas as pd
5
6 # Sweep Llama 3 8B across multiple batch sizes on an H100 GPU
7 model = Models.Language.Llama3_8B
8 hardware = Hardware.Cloud.H100
9
10 results = []
11 for batch_size in [1, 8, 32, 128, 256]:
12     # Engine.solve() wraps the Tier 1 SingleNodeModel
13     perf = Engine.solve(model, hardware, batch_size=batch_size, precision="fp16")
14
15     results.append({
16         "Batch Size": batch_size,
17         "Throughput (tok/s)": round(perf.throughput.magnitude),
18         "Bottleneck": perf.bottleneck,
19         "MFU": round(perf.mfu, 3)
20     })
21
22 df = pd.DataFrame(results)
23 print(df)
24 #   Batch Size Throughput (tok/s) Bottleneck   MFU
25 # 0           1           178      Memory 0.003
26 # 1           8           893      Memory 0.014

```


27	# 2	32	1564	Memory	0.025
28	# 3	128	1925	Memory	0.031
29	# 4	256	2002	Memory	0.033

Listing 6: **Programmatic Sweeps (Tier 1).** Using the Engine to sweep a workload across a hardware constraint space (batch size). Throughput rises from 178 to $\sim 2,000$ tokens/s and then saturates: per-request memory traffic grows with the batch, so decode stays memory-bound at every point. Because execution takes <1 ms per point, sweeping hundreds of configurations is trivial.

A.4.2 Scenario B: Cross-Domain Carbon Accounting (Tier 1 Composition)

Scenario: You have been tasked with estimating the carbon footprint of training a 70B parameter model. The listing first uses `DistributedModel` to compute the step latency for a registry-backed 256-GPU fleet, then converts a one-trillion-token target into a training duration. That duration becomes the input to `SustainabilityModel`, where the Iowa datacenter profile supplies grid intensity and facility overhead; the composition works because solver outputs are dimensioned quantities rather than untyped floats.

```

1 from mlsysim.solvers import DistributedModel, SustainabilityModel
2 from mlsysim.systems.registry import Systems
3 from mlsysim.infrastructure.registry import Infrastructure
4 from mlsysim.models.registry import Models
5
6 fleet = Systems.Clusters.Research_256 # 32x DGX H100 nodes
7
8 # 1. Performance Domain: how long does one training step take?
9 perf = DistributedModel().solve(
10     model=Models.Language.Llama3_70B, fleet=fleet,
11     batch_size=1024, seq_len=2048, tp_size=8, pp_size=1,
12     activation_recomputation=True,
13 )
14
15 # 2. Train for 1T tokens: convert step latency into a duration
16 steps = 1e12 / (1024 * 2048)
17 training_days = (perf.step_latency_total * steps).to("day").magnitude
18
19 # 3. Sustainability Domain: what is the carbon impact?
20 sust = SustainabilityModel().solve(
21     fleet=fleet, duration_days=training_days,
22     datacenter=Infrastructure.Datacenters.Iowa_Reference, # high-carbon grid
23     mfu=0.42,
24 )
25 print(f"Training Duration: {training_days:.0f} days")
26 # Training Duration: 57 days
27 print(f"Total Carbon: {sust.carbon_footprint_kg / 1000:.1f} tonnes CO2e")
28 # Total Carbon: 110.3 tonnes CO2e

```

Listing 7: **Resolver Composition (Tier 1).** Output quantities from one solver (the computed step latency from the `DistributedModel`) feed directly into subsequent solvers (the `SustainabilityModel`), enabling end-to-end full-stack analysis without manual unit conversion.

A.4.3 Scenario C: Uncovering the Data Wall (Tier 1)

Scenario: Your GPU utilization is mysteriously low during a computer vision training job. The code splits the diagnosis into two checks: `DataModel` compares the required image stream against the node's I/O path, while `TransformationModel` compares CPU preprocessing throughput against accelerator step time. Reading the two results together distinguishes a storage or transfer bottleneck from a host-side decode and augmentation bottleneck.

```

1 from mlsysim.solvers import DataModel, TransformationModel
2 from mlsysim.hardware.registry import Hardware
3 from mlsysim.core.units import Q_
4
5 # 1. Define the ingestion demand (e.g., training a vision model)
6 # We need to feed 40,000 images per second to the GPU.

```

```

7 demand_rate = Q_("40000 1/s") * Q_("150 KB") # ~6 GB/s
8
9 # 2. Check if the A100 node's I/O path can handle the bandwidth
10 data_result = DataModel().solve(
11     workload_data_rate=demand_rate,
12     hardware=Hardware.Cloud.A100
13 )
14 print(f"I/O Stalled: {data_result.is_stalled}") # False
15
16 # 3. Check if 16 CPU workers can decode + augment fast enough
17 # (850 images/s per worker, 150 KB per image)
18 cpu_throughput = 16 * 850 * Q_("150 KB/s")
19 transform_result = TransformationModel().solve(
20     batch_size=2048,
21     sample_size_bytes=Q_("150 KB"),
22     cpu_throughput=cpu_throughput,
23     accelerator_step_time=Q_("48 ms"),
24 )
25 print(f"CPU Stalled: {transform_result.is_bottleneck}") # True
26 print(f"Accel. Utilization: "
27       f"{transform_result.accelerator_utilization:.0%}") # 32%

```

Listing 8: Data Pipeline Analysis (Tier 1). Evaluating the “Data Wall”. This example separates I/O bandwidth bottlenecks (storage to GPU) from CPU transformation bottlenecks (decoding and augmenting data), in the spirit of Case Study S2 but with an under-provisioned 16-worker pipeline.

A.4.4 Scenario D: SLA-Driven Hardware Synthesis (Tier 2)

Scenario: You are provisioning hardware for a new LLM application. You do not start with a GPU SKU; you start with the business requirement that the application should emit a token every 30 milliseconds. The `SynthesisSolver` inverts the same first-order Roofline model used for evaluation: given `Models.Language.Llama3_8B`, the target latency, and precision, it returns the memory-bandwidth and compute-throughput requirements that candidate hardware must clear.

```

1 from mlsysim.solvers import SynthesisSolver
2 from mlsysim.models.registry import Models
3 from mlsysim.core.units import Q_
4
5 # We want to serve Llama-3 8B.
6 # Our strict SLA dictates an inter-token latency of 30 ms.
7 # What is the minimum hardware capability required to achieve this?
8
9 solver = SynthesisSolver()
10 requirements = solver.solve(
11     model=Models.Language.Llama3_8B,
12     target_latency=Q_("30 ms"),
13     precision="fp16"
14 )
15
16 # Output the hardware requirements
17 print(f"Required HBM Bandwidth: {requirements.required_bw.to('GB/s'):.1f}")
18 # Required HBM Bandwidth: 535.3 GB / second
19 print(f"Required Compute: {requirements.required_flops.to('TFLOPs/s'):.1f}")
20 # Required Compute: 1.1 TFLOPs / second

```

Listing 9: SLA-Driven Hardware Synthesis (Tier 2). Instead of evaluating a known piece of hardware, the `SynthesisSolver` works backward. Given a strict latency SLA, it algebraically inverts the Roofline equations to output first-order minimum memory bandwidth and compute throughput requirements for procurement screening.

A.4.5 Scenario E: Speculative Decoding Speedup (Tier 1)

Scenario: You are serving a large frontier model and want to know how much faster inference would be if you used a smaller, cheaper draft model to speculatively decode tokens. The first call establishes the autoregressive baseline

for the target model on the same H100 supply point. The second call adds a draft model and an acceptance-rate assumption, allowing `ServingModel` to account for draft generation and target-model verification before reporting the effective inter-token latency and speedup.

```
1 from mlsysim.solvers import ServingModel
2 from mlsysim.hardware.registry import Hardware
3 from mlsysim.models.registry import Models
4
5 # Serve Llama 3.1 70B using Llama 3.1 8B as a draft model
6 target_model = Models.Language.Llama3_70B
7 draft_model = Models.Language.Llama3_8B
8 hardware = Hardware.Cloud.H100
9
10 solver = ServingModel()
11
12 # Standard Autoregressive Decoding
13 base_result = solver.solve(target_model, hardware, seq_len=2048, batch_size=1)
14 print(f"Standard ITL: {base_result.itl.to('ms'):.2f}")
15 # Standard ITL: 43.15 millisecond
16
17 # Speculative Decoding
18 spec_result = solver.solve(
19     target_model, hardware, seq_len=2048, batch_size=1,
20     draft_model=draft_model,
21     draft_acceptance_rate=0.75
22 )
23 print(f"Speculative ITL: {spec_result.itl.to('ms'):.2f}")
24 # Speculative ITL: 20.53 millisecond
25 print(f"Speedup: {(base_result.itl / spec_result.itl).magnitude:.2f}x")
26 # Speedup: 2.10x
```

Listing 10: Speculative Decoding Analysis (Tier 1). The `ServingModel` natively supports algorithmic optimizations like speculative decoding. By providing a draft model and an expected acceptance rate, the solver calculates the effective inter-token latency (ITL) by modeling the compute-bound draft phase and the memory-bound verification phase.

A.4.6 Scenario F: The Full-Stack Scorecard (Tier 2)

Scenario: Rather than invoking individual solvers, you want the single “will it run, is it fast enough, and what does it cost” verdict for a deployment. An executable `Scenario` is a named bundle of workload, target system, SLA, and operating context. Calling `evaluate()` runs the Feasibility, Performance, and Macro levels in order, short-circuiting impossible cases and returning a `SystemEvaluation` whose scorecard preserves the reason for each pass or failure.

```
1 import mlsysim
2
3 # A pre-built executable scenario: chatbot serving with an SLA + grid context
4 evaluation = mlsysim.Scenarios.ChatbotServing.evaluate()
5 print(evaluation.scorecard())
6 # Level 1: Feasibility [PASS] Model fits in memory (16.1 GB / 85.9 GB)
7 # Level 2: Performance [PASS] Latency: 5.60 ms (Target: 500 ms)
8 # Level 3: Macro [PASS] Annual Carbon: 2946.3 kg | TCO: $10,407
9 print(evaluation.passed_all) # True
```

Listing 11: Three-Level Scorecard (Tier 2). `Scenario.evaluate()` composes the resolver chain automatically and returns a `SystemEvaluation`: Level 1 checks memory feasibility, Level 2 reports latency against the SLA, and Level 3 reports cost and carbon.

A.4.7 Scenario G: Automated Parallelism Search (Tier 3)

Scenario: You have a fixed cluster and a large model and want the framework to *find* the best 3D-parallelism split rather than specifying it by hand. The Tier 3 `ParallelismOptimizer` enumerates power-of-two $TP \times PP \times DP$ factorizations of the available devices, computes per-GPU training state for each candidate, and discards layouts that

exceed HBM. The result keeps the best MFU configuration and a ranked candidate list, making the search procedure visible rather than a black-box recommendation.

```

1 import mlsysim
2 from mlsysim.solvers import ParallelismOptimizer
3
4 fleet = mlsysim.Systems.Clusters.Research_256 # 256x H100
5 result = ParallelismOptimizer().solve(
6     mlsysim.Models.Language.Llama3_70B, fleet,
7     batch_size=2048, precision="fp16", efficiency=0.5,
8 )
9 print(result.best_config) # {'tp': 8, 'pp': 2, 'dp': 16}
10 print(f"MFU: {result.best_mfu:.3f}") # MFU: 0.315
11 for c in result.top_candidates[:3]:
12     print(c["config"], f'{c["mfu"]:.3f}')
13 # {'tp': 8, 'pp': 2, 'dp': 16} 0.315
14 # {'tp': 4, 'pp': 4, 'dp': 16} 0.274
15 # {'tp': 8, 'pp': 4, 'dp': 8} 0.272

```

Listing 12: Parallelism Optimizer (Tier 3). The optimizer enumerates valid TP/PP/DP factorizations of the cluster, discards configurations whose per-GPU training state exceeds HBM, and returns the highest-MFU survivor with its ranked candidate list.

A.4.8 Scenario H: From Datacenter to Microcontroller (Tier 1)

Scenario: The same dimensionally strict engine that sizes an H100 fleet also sizes a microcontroller. The loop changes only the registry-backed model and hardware entries; `Engine.solve()` still lowers the workload demand, reads the supply constraints, and returns feasibility, bottleneck regime, latency, and per-inference energy. This is the benefit of the demand–supply contract: datacenter and TinyML examples share the same code path while drawing facts from very different parts of the registry.

```

1 from mlsysim.engine.engine import Engine
2 from mlsysim.hardware.registry import Hardware
3 from mlsysim.models.registry import Models
4
5 deployments = [
6     (Models.Vision.LeNet1, Hardware.Tiny.ESP32_S3),
7     (Models.Vision.MobileNetV2, Hardware.Edge.JetsonOrinNano),
8     (Models.Vision.MobileNetV2_Alpha0_5, Hardware.Edge.Coral),
9 ]
10 for model, hw in deployments:
11     perf = Engine.solve(model, hw, batch_size=1, precision="int8")
12     print(f"{hw.name:24s} {perf.bottleneck}-bound "
13           f"{perf.latency.to('ms'):.2f} {perf.energy.to('mJ'):.2f}")
14 # ESP32-S3 (AI) Compute-bound 1.03 millisecond 0.13 millijoule
15 # NVIDIA Jetson Orin Nano Memory-bound 0.80 millisecond 3.66 millijoule
16 # Google Coral Edge TPU Memory-bound 1.28 millisecond 0.77 millijoule

```

Listing 13: Edge and TinyML Deployment (Tier 1). The identical `Engine.solve` call evaluates milliwatt-class devices: a LeNet variant fits in the ESP32-S3’s on-chip SRAM, while MobileNet variants run on a Jetson Orin Nano and a Coral Edge TPU. Energy is per inference.